

Technische
Universität
Dresden



Distributed Computing II

Digitization and Data Analytics

Structure of the Lecture Series

Lecture 1: Introduction

Lecture 2: Data Preprocessing

Lecture 3: Research Data Management

Lecture 4: Data Analytics and Statistics

Lecture 5: Machine Learning I

Lecture 6: Machine Learning II

Lecture 7: Machine Learning III

Lecture 8: Deep Learning I

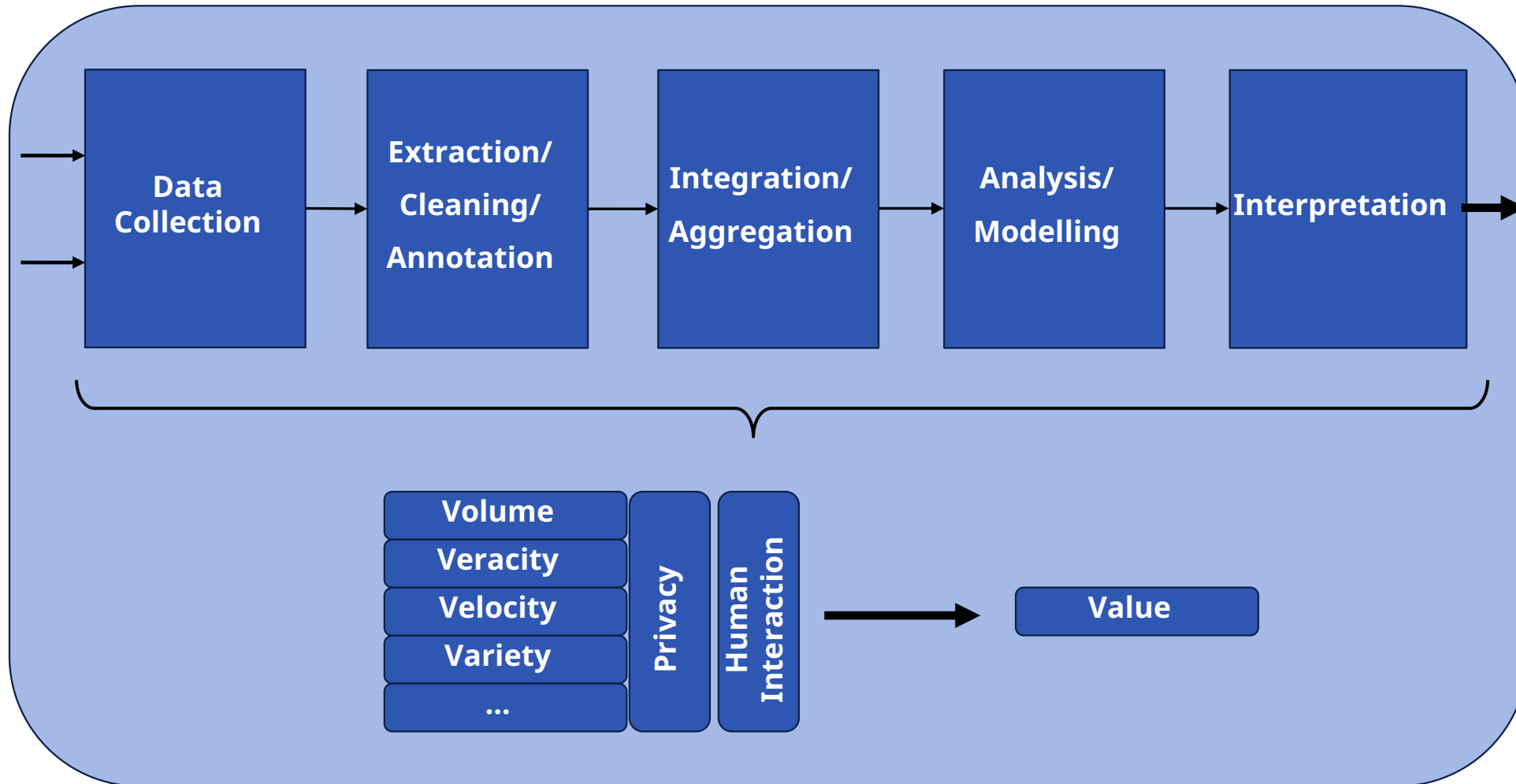
Lecture 9: Deep Learning II

Lecture 10: Distributed Computing I

Lecture 11: Distributed Computing II

Lecture 12: Consequences

Short recap: Data Processing Pipeline

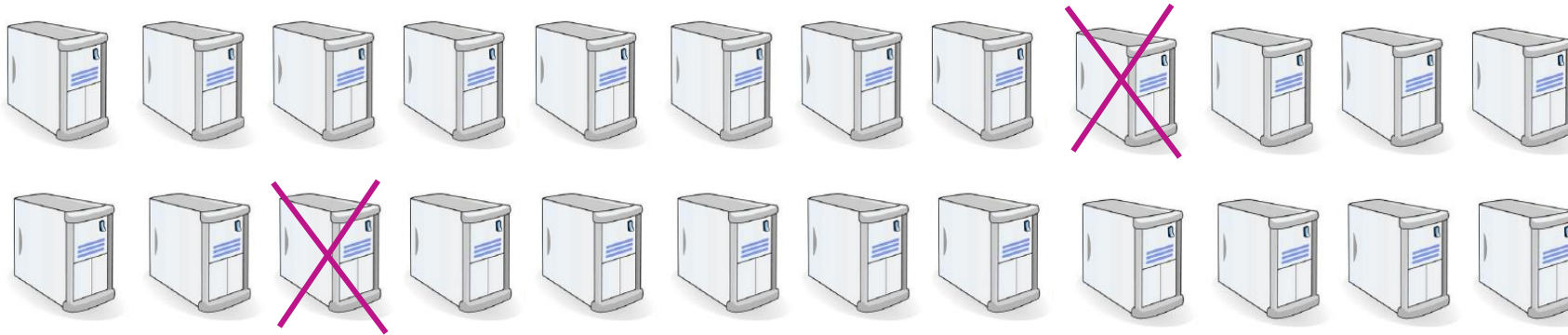


Short recap: Data Processing and Analytics

Why not just taking 'low-cost' hardware like standard server components?

Scale out paradigm

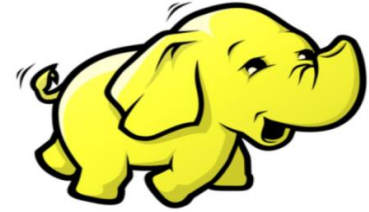
Many machines (hundreds, thousands)



As opposed to scale-up, where one very powerful (single) server is used

Do not rely on each instance!

MapReduce Paradigm



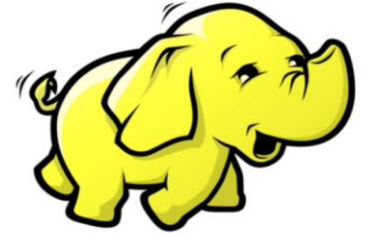
How to process and store data on this infrastructure?

Scale out paradigm:

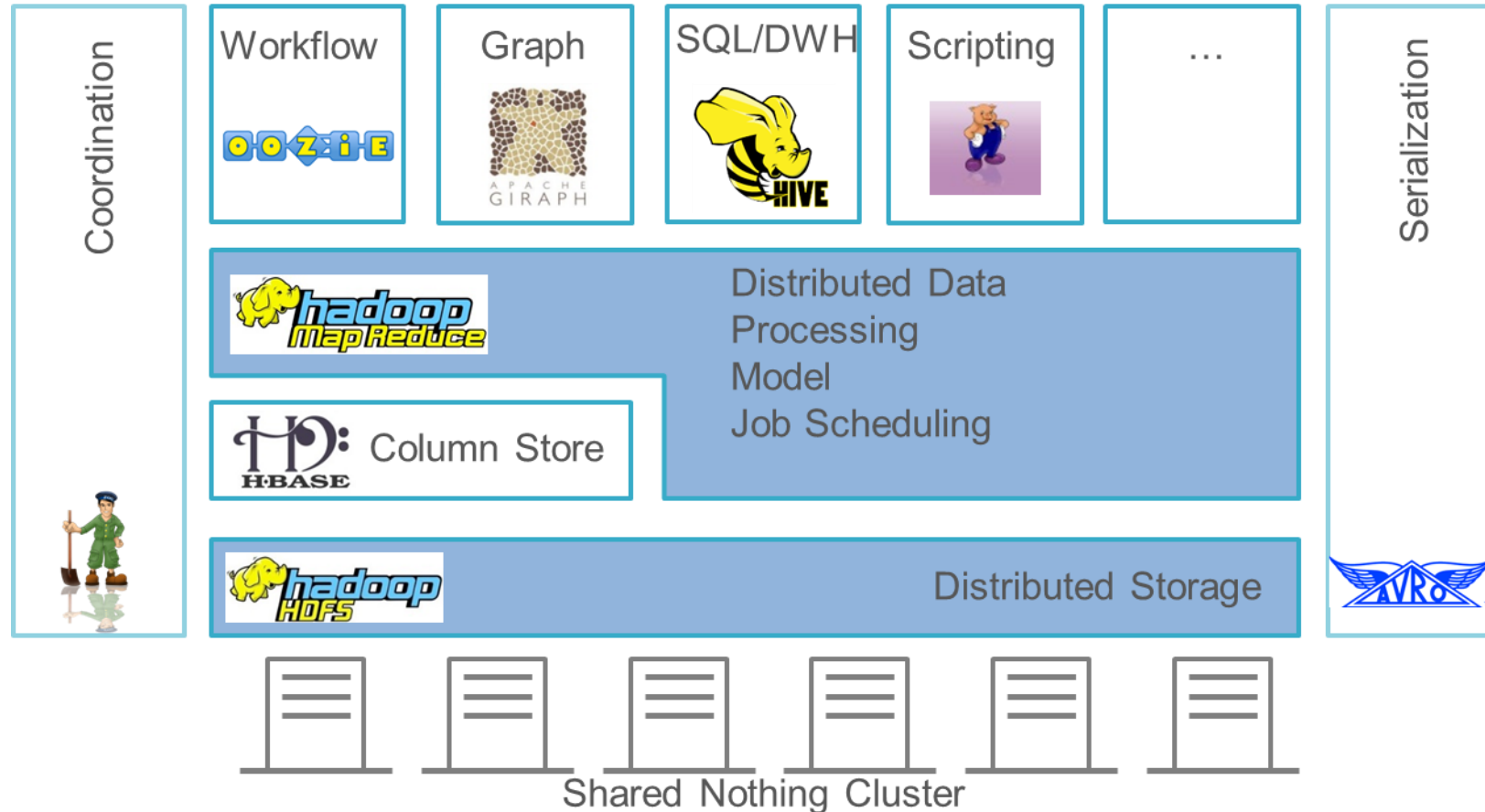
- The Google File System (**Sanjay Ghemawat**, H. Gobioff, S. Leung. SOSP 2003)
- MapReduce: Simplified Data Processing on Large Clusters (**Jeff Dean, S. Ghemawat**. OSDI 2004)
- Hadoop is **open-source** software framework
- Supports data-intensive **distributed** applications and clones the Google's MapReduce
- Designed to process very **large amounts** of unstructured and complex **data**
- Designed to run on a **large** number of machines



Hadoop 1.0 Ecosystem

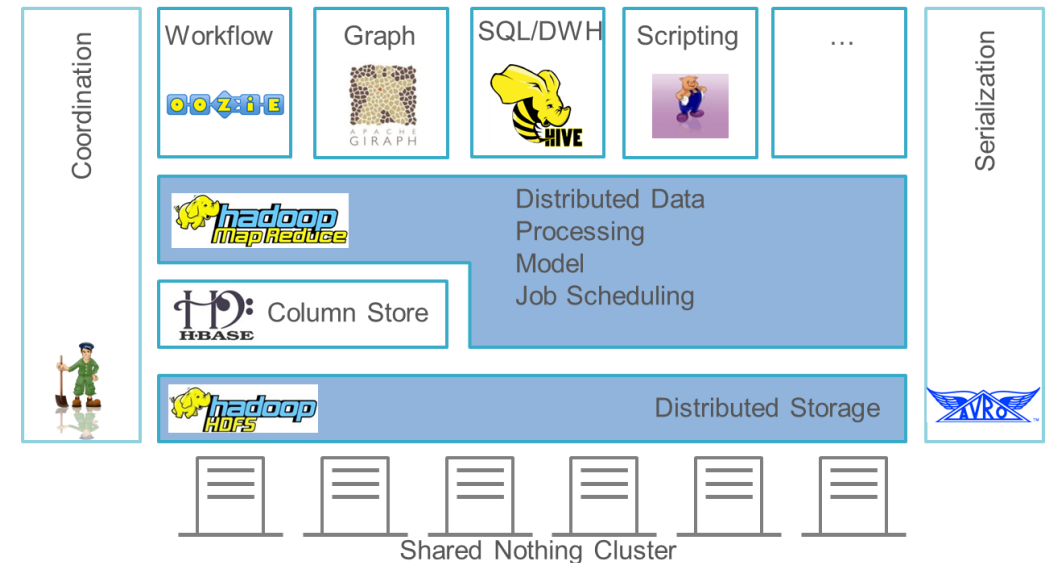
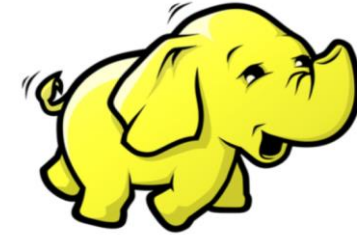


Not just simple functions – towards interoperable frameworks

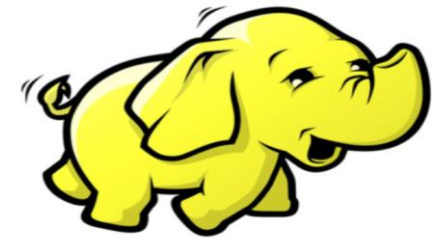


Assumptions and Goals

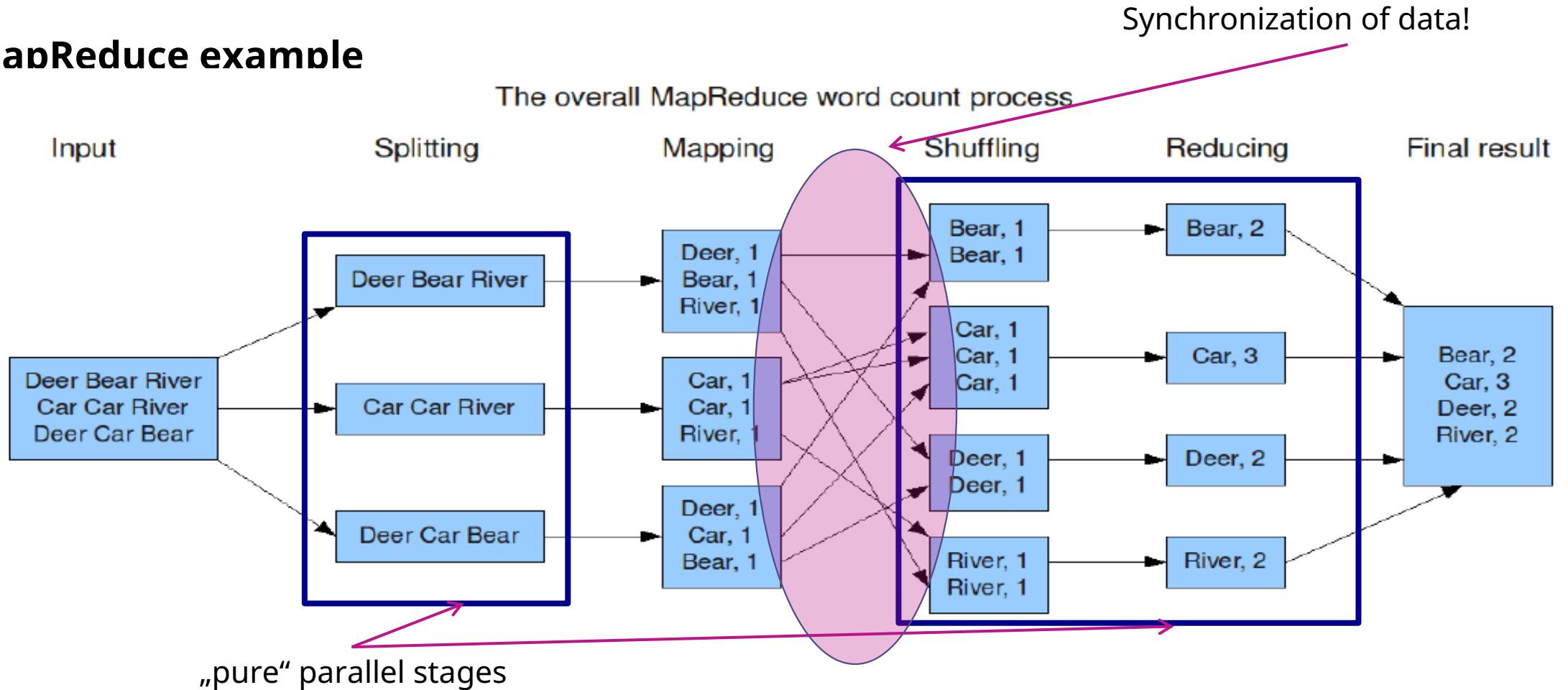
- Hardware Failure
 - Hundreds to thousands of commodity servers per cluster
 - Failure is the norm rather than the exception
- Streaming Data Access
 - Applications need streaming access to the data
 - Batch processing rather than interactive use
- Large Data Sets
 - Tens of millions of files each with gigabytes to terabytes in size
- Moving computation to data
 - Computation „near“ the data it operates on
 - Knowledge about data location is needed
- Portability
 - Run on heterogeneous hardware and software platforms



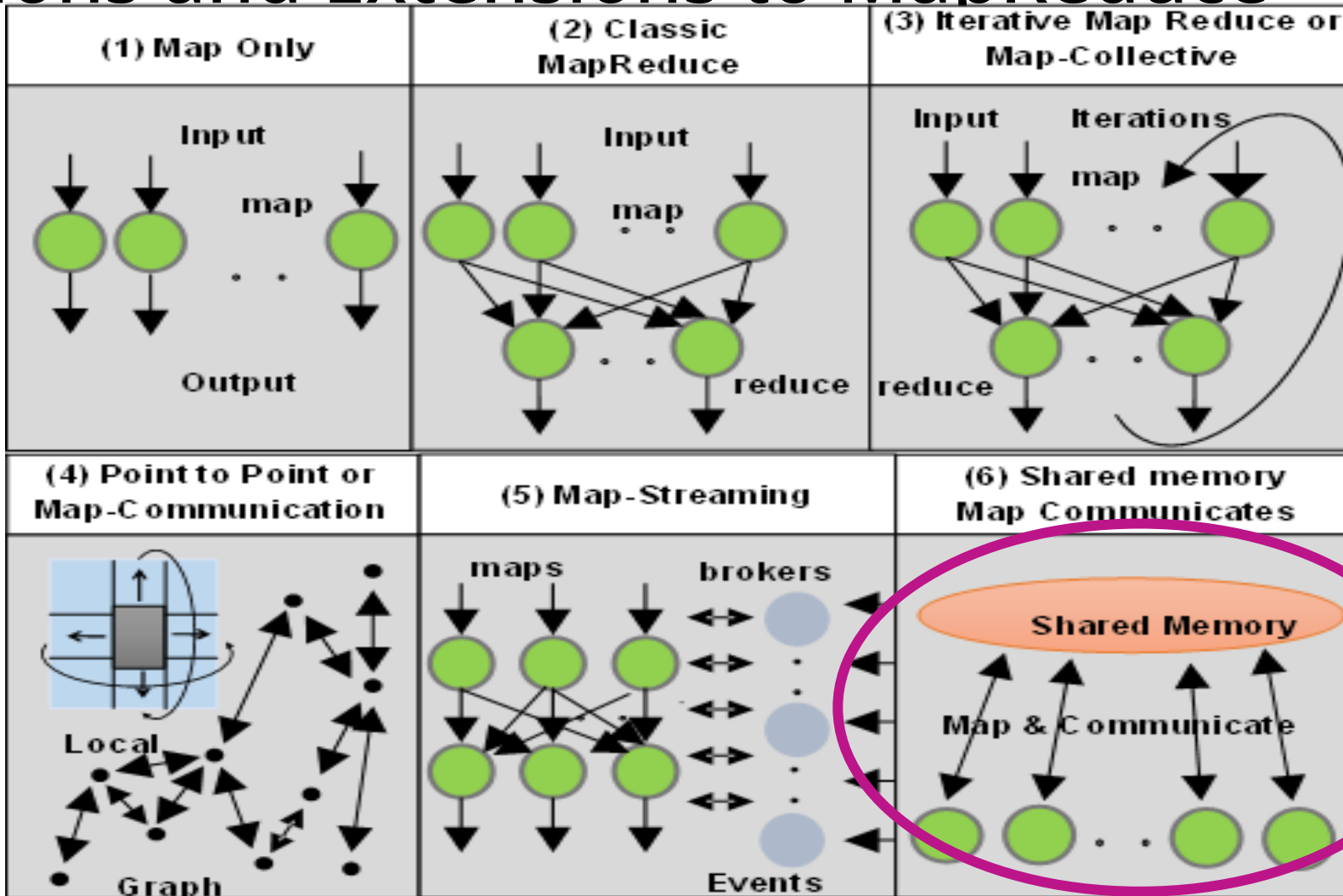
Hadoop 1.0 Ecosystem



MapReduce example



Limitations and Extensions to MapReduce



In-Memory
Processing

[Geoffrey Fox: Lecture: Building a Library at the Nexus of High Performance Computing and Big Data, Indiana University]

Data driven analysis – „Big Data 2.0“

In-Memory Processing

- **Previously:**

- processing based on disk storage and relational databases
- using SQL query language, but became too slow with increasing data

- **Now:**

- **processing** of data stored in **in-memory** (database)
- towards data analysis in real time; lifts limitations of IO-waiting times (in IO-intense applications usually show stopper)
- RAM has come down in cost, in-memory computing becomes available in larger scale
- provides basis for other algorithms pipelines, not just separating compute tasks: Iterations, Joins

- **History:**

- triggered by business intelligence (BI) needs
BI: analytic reports would be started and results would come later (i.e. "coffee break analytics")

<https://www.hpe.com/de/de/what-is/in-memory-computing.html>

In-Memory (Hardware) Processing

Caching

- E.g. Processor caches, file cache, disk cache, permission cache

Replication

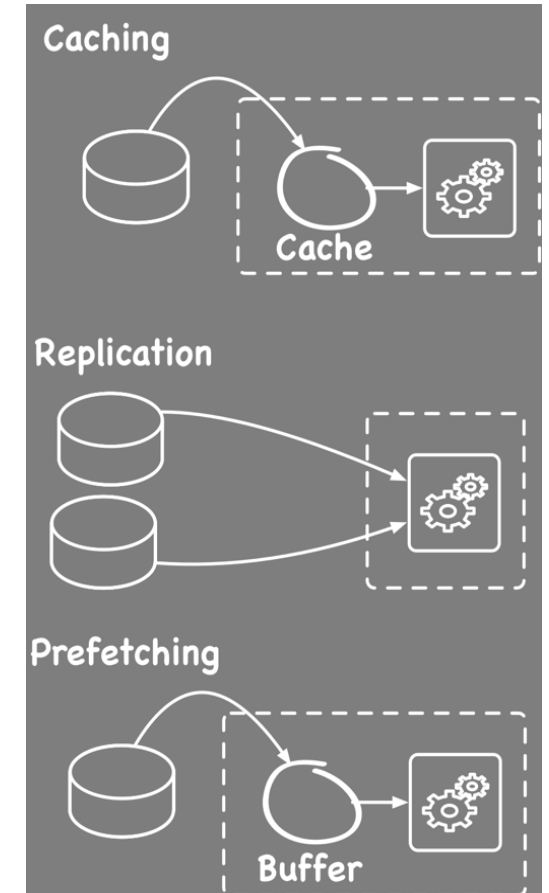
- E.g. RAID, Content Distribution Networks (CDN), Web/Server Caches

Prefetching – “predict” which data will be needed and pre-fetched

- Tradeoff bandwidth
- E.g. disk caches, Google Earth

Limitations:

- Caching works only if working set is small (access to data copies)
- Prefetching only works when access patterns are predictable
- Replication is expensive and limited by receiving side machines
- Application behavior (efficiency) depends on actual workflow

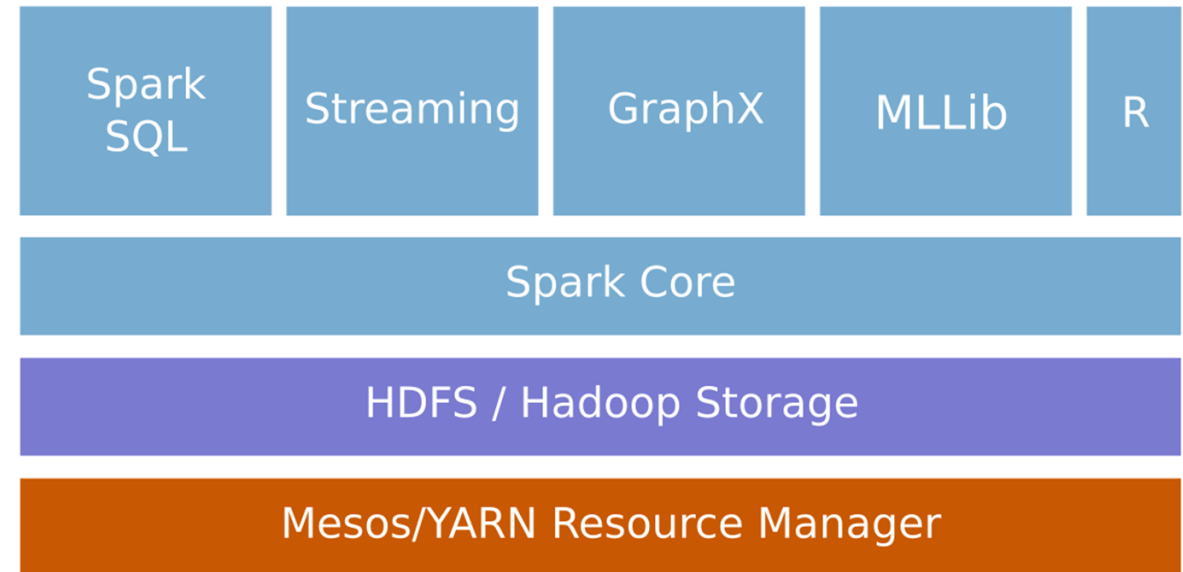


Apache Spark

Third generation – multi purpose frameworks and large diversity of analytics environments, e.g.:

Apache Spark

- A fast and general engine for large scale data processing
- Created by AMPLab (Berkeley/2009) now Databricks
- Written in Scala and licensed under Apache Foundation
- Strong international developer community and organized as open source

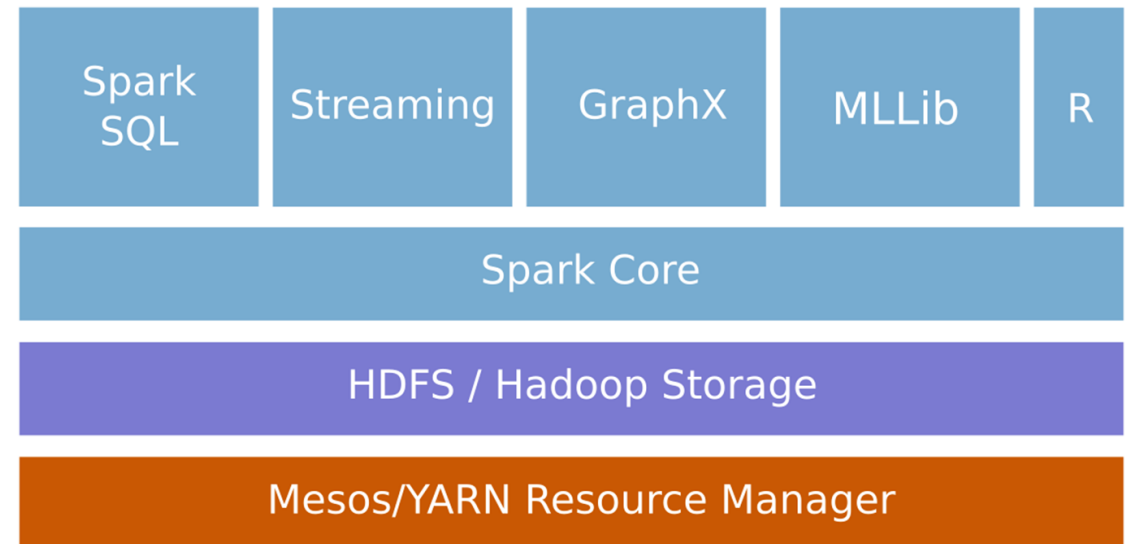


Apache Spark

Offers stack of libraries for **analytics** including SQL and DataFrames, MLlib for **machine learning**, **GraphX**, and Spark **Streaming**

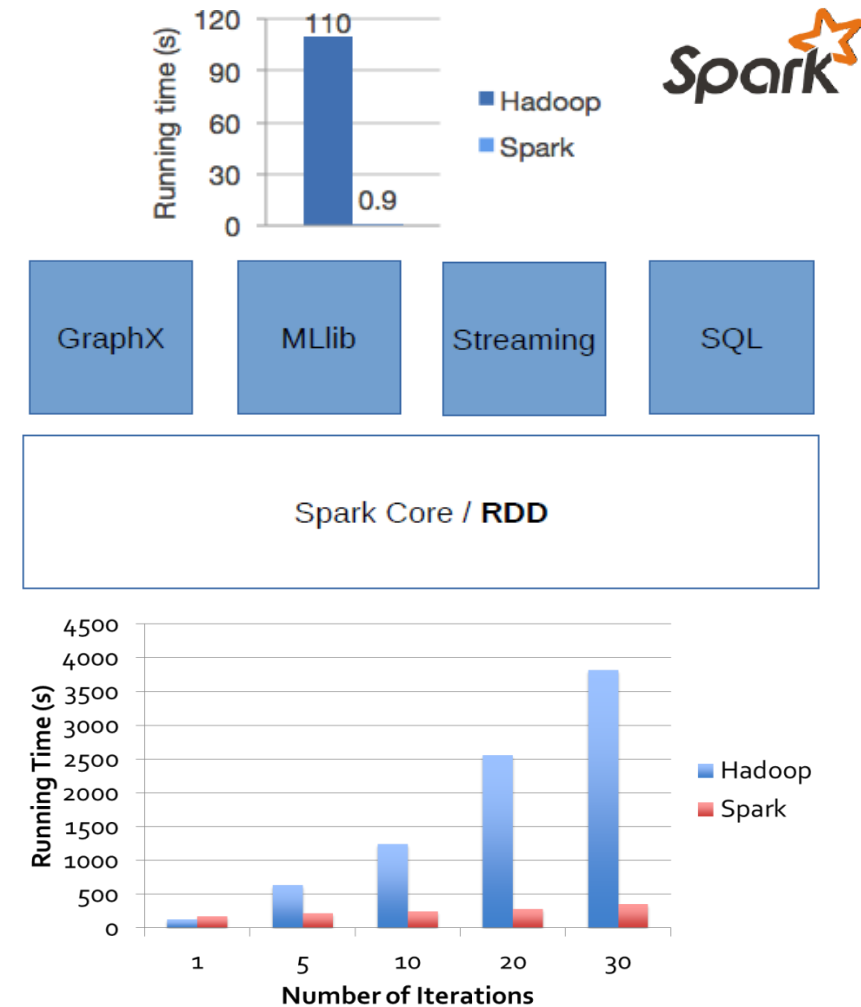
Data access can be realized via HDFS, Cassandra, HBase, Hive, Tachyon, and any Hadoop data source

Run Spark standalone or in cluster mode, on EC2, on Hadoop YARN, or on Apache Mesos



Spark overview

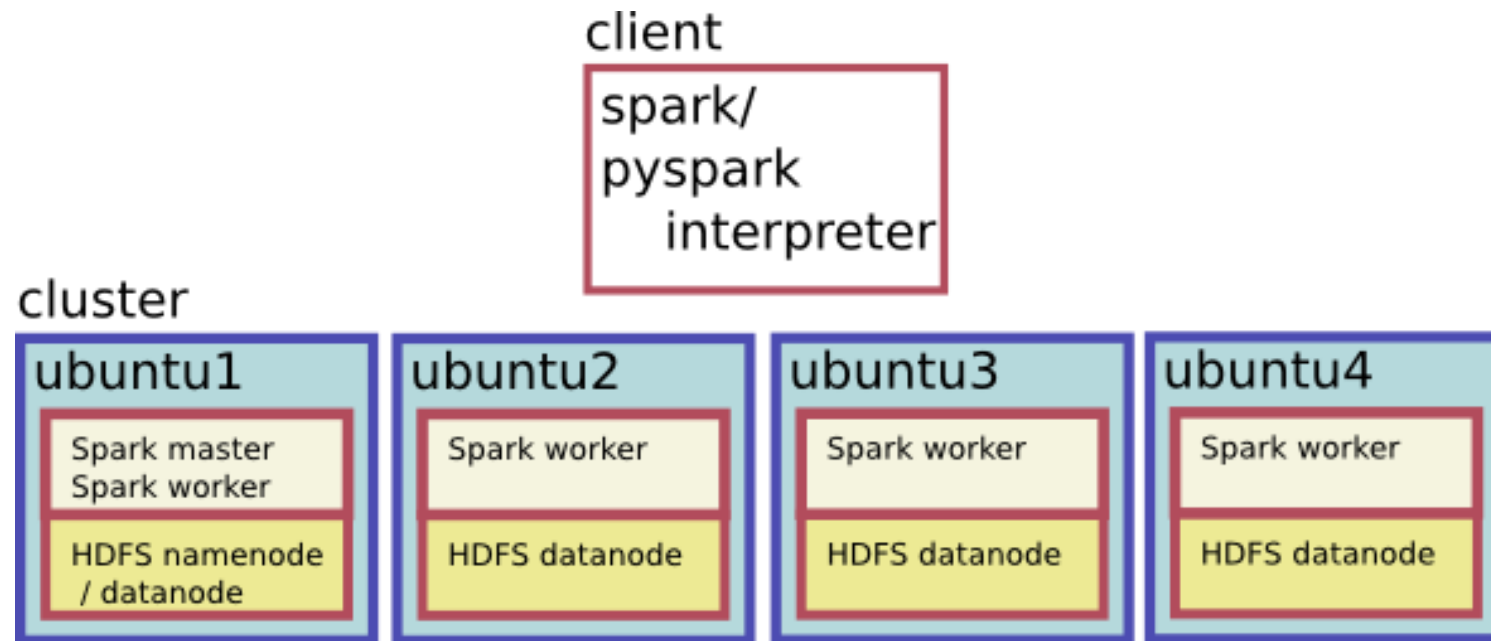
- Apache Spark offers Directed Acyclic Graph (DAG) execution engine that supports cyclic data flow and in-memory computing
- Offers over 80 high-level operators to build parallel applications use it interactively from the Scala, Python and R shells
- Offers stack of libraries including SQL and DataFrames, MLlib for machine learning, GraphX, and Spark Streaming
- Combine libraries seamlessly in the same application



@ Spark Homepage

Spark overview

Run Spark standalone or in cluster mode, on EC2, on Hadoop YARN, or on Apache Mesos
Access data in HDFS, Cassandra, HBase, Hive, Tachyon, and any Hadoop data source



Differences between HPC and Big Data scheduling

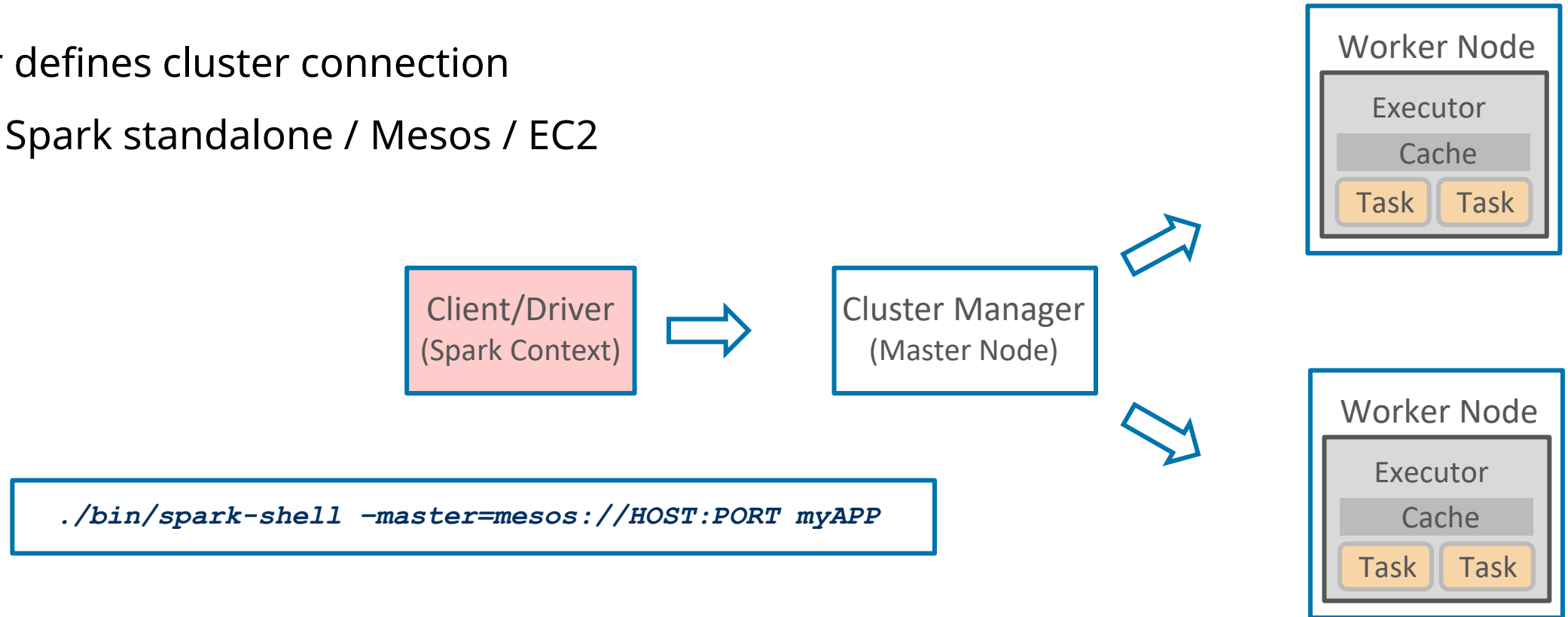
- HPC
 - requires fine-grained control of what resources (cores, accelerators, memory, time, etc.)
 - resources are scheduled with tools like Slurm or others like Grid Engine, Moab, LoadLeveler, etc.
- BD: e.g. oldest framework Hadoop:
 - Hadoop (version 1) has an integrated scheduler that consists of a master Job Tracker
 - communicates with Task Trackers on the nodes
 - MapReduce work (basics of Hadoop) is supervised by the Job Tracker
 - Fail-save mechanism: can manage failure through the Job Tracker
- Programming differences
 - HPC: typically C/C++, Fortran; both SIMD/MIMD supported; nowadays Python support
 - BD: Java, Scale, Python
 - MapReduce can be classified as a SIMD (single-instruction, multiple-data) problem

Under the hood – Spark-Context

Connection of the client to the available cluster

Master defines cluster connection

Local / Spark standalone / Mesos / EC2



Under the hood – Spark-Context

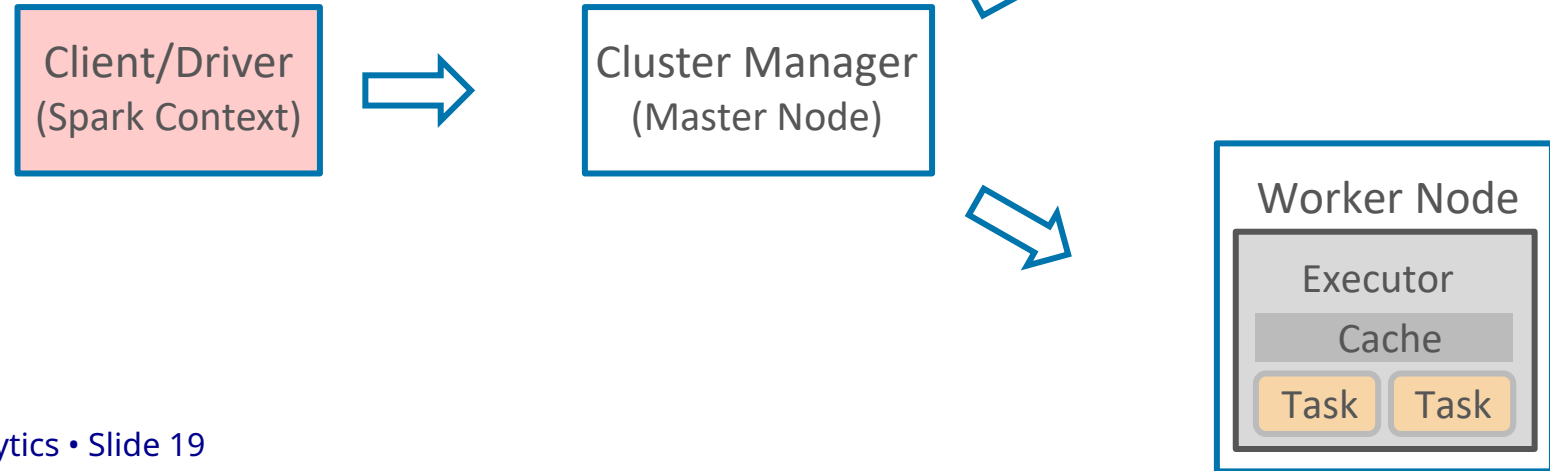
Workflow

Connects to a cluster manager which allocates resources across applications

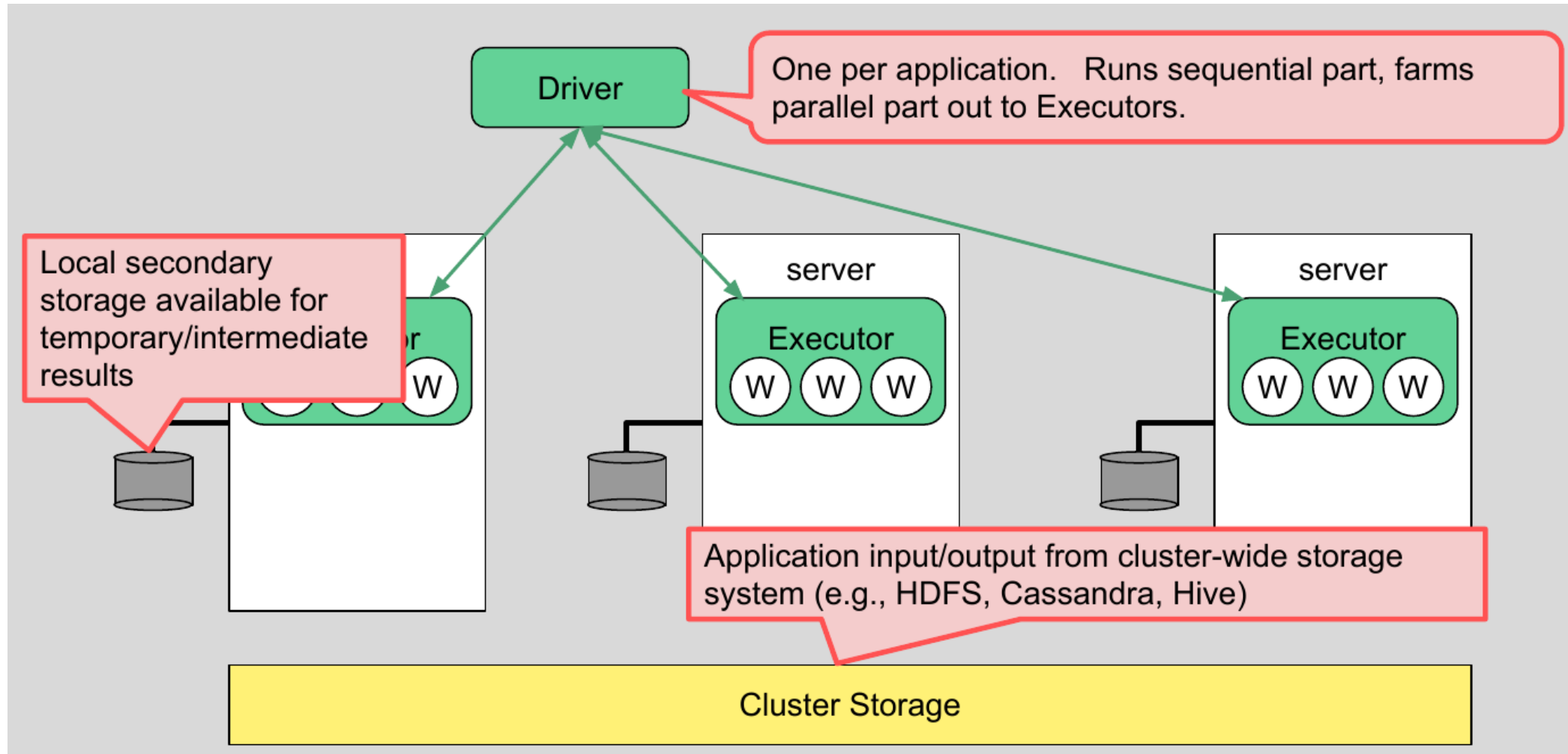
Acquires executors on worker nodes to process computations on stored data

Sends driver application code to executors

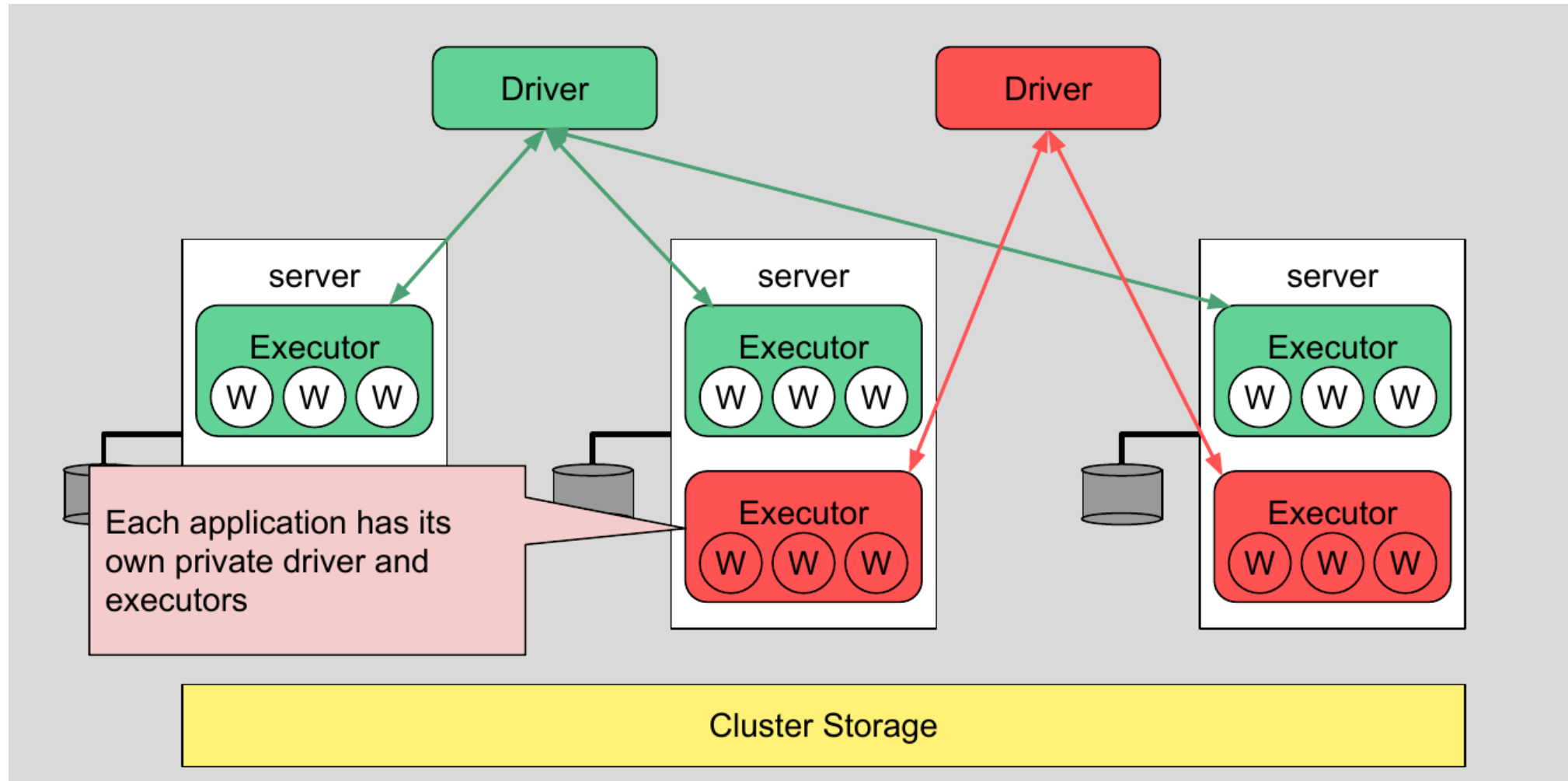
Distributes tasks for the executors to run separately



Spark cluster



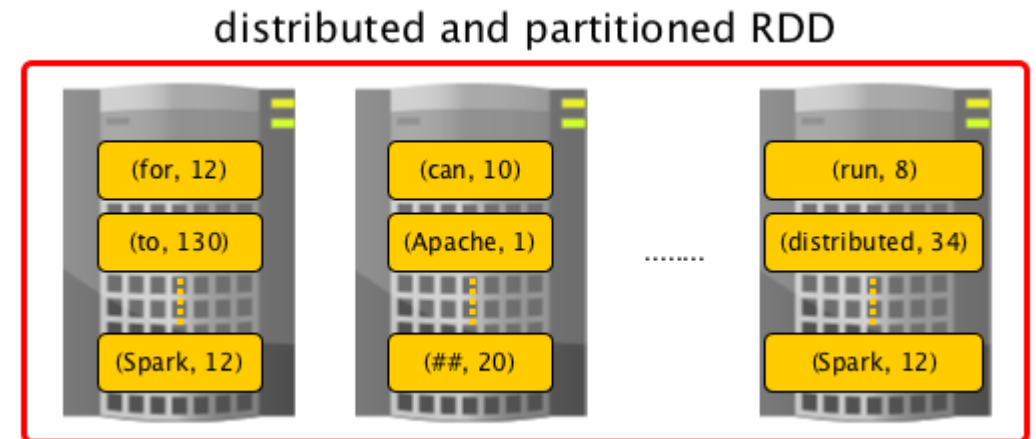
Spark cluster



Data Structure from Developer's point of View

RDD – Resilient Distributed Dataset

- RDD is a resilient and distributed collection of records
- **Resilient**, i.e. fault-tolerant with the help of RDD lineage graph and so able to re-compute missing or damaged partitions due to node failures
- **Distributed** with data residing on multiple nodes in a cluster
- **Dataset** is a collection of partitioned data with primitive values or values of values, e.g. tuples or other objects



Data Structure from Developer's point of View

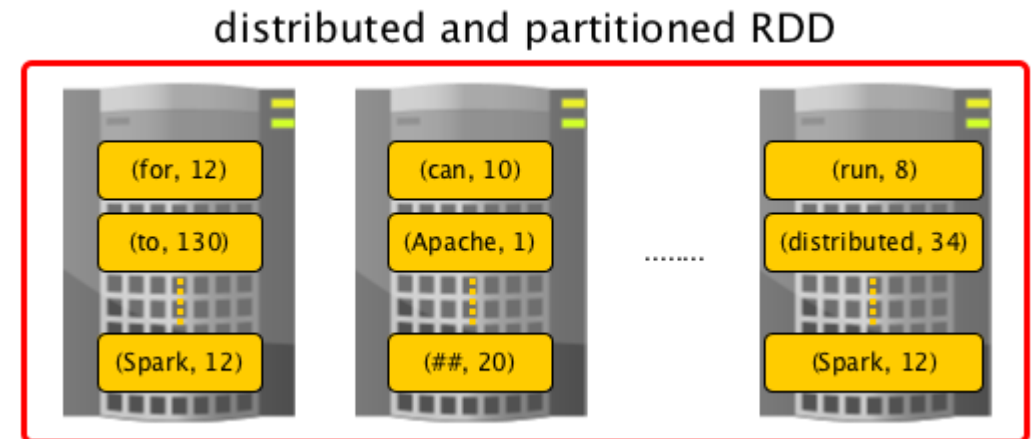
RDD – Resilient Distributed Dataset

Parallelism controlled by RDD using 'repartition' method

Spark tries to **distribute computing** to already partitioned data, **avoiding data exchange**

RDDs support two kinds of **operations**:

- **transformations** - lazy operations that return another RDD
- **actions** - operations that trigger computation and return values



Apache Spark

RDD

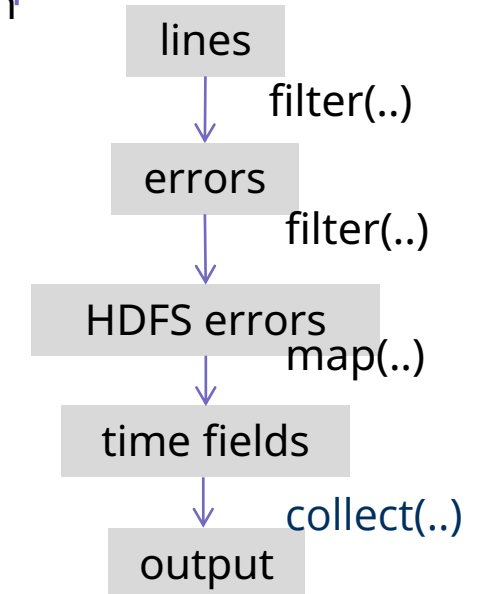
```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
errors.filter(_.contains("HDFS")).map(_.split('\t')(3)).collect()
```

Transformation

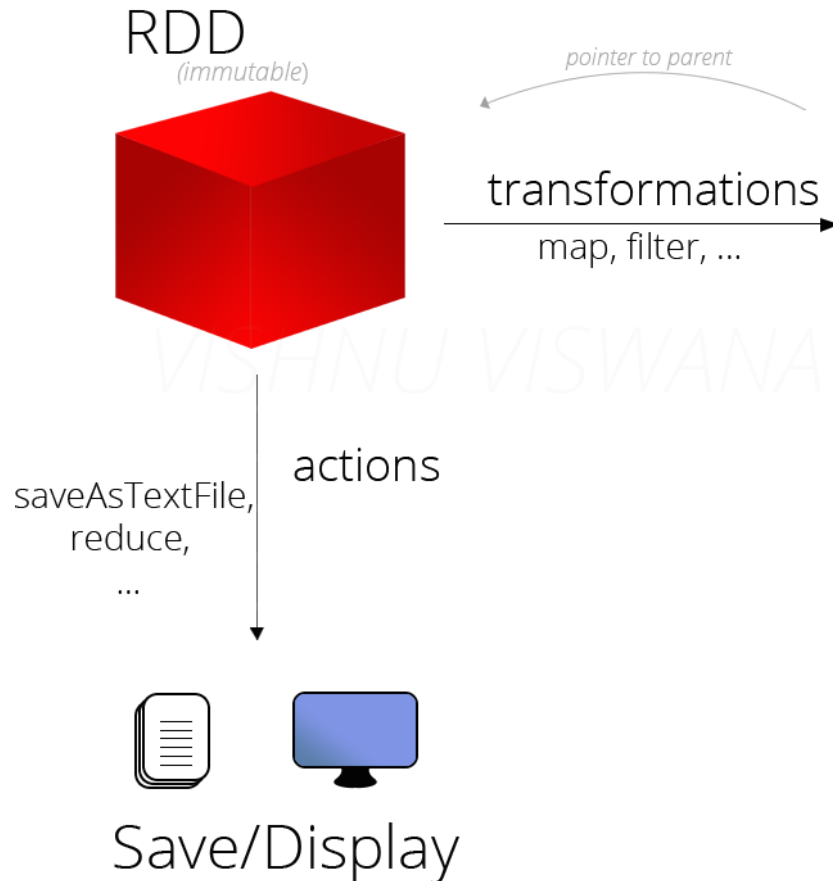
Action

Apache Spark offers Directed Acyclic Graph (DAG) execution engine that supports cyclic data flow and in-memory computing

RDD: Resilient Distributed Dataset
Sparks core concept



RDD – Resilient Distributed Dataset



New RDD

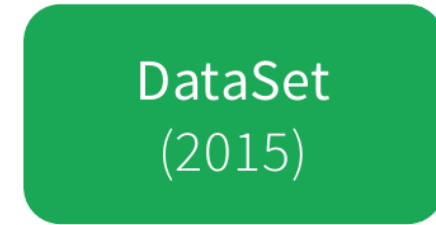
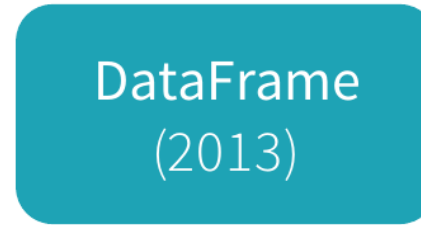
Transformations

<http://spark.apache.org/docs/latest/rdd-programming-guide.html#transformations>

Actions

<http://spark.apache.org/docs/latest/rdd-programming-guide.html#actions>

Data Structure Evolution



- Distribute collection of JVM objects
- Functional Operators (map, filter, etc.)

- Distribute collection of Row objects
- Expression-based operations and UDFs
- Logical plans and optimizer
- Fast/efficient internal representations

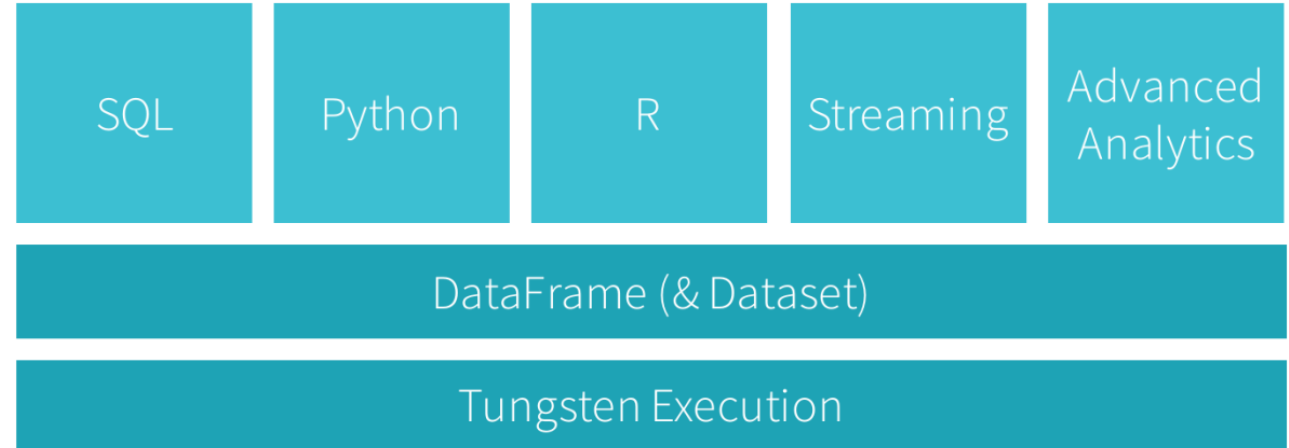
- Internally rows, externally JVM objects
- “Best of both worlds”
type safe + fast

UDF = user defined functions

Jump Start into Apache® Spark™ and Databricks; Denny Lee @Databricks

RDD to DataFrames to Datasets

Enhanced stack:



[Jump Start into Apache® Spark™ and Databricks](#); Denny Lee @Databricks

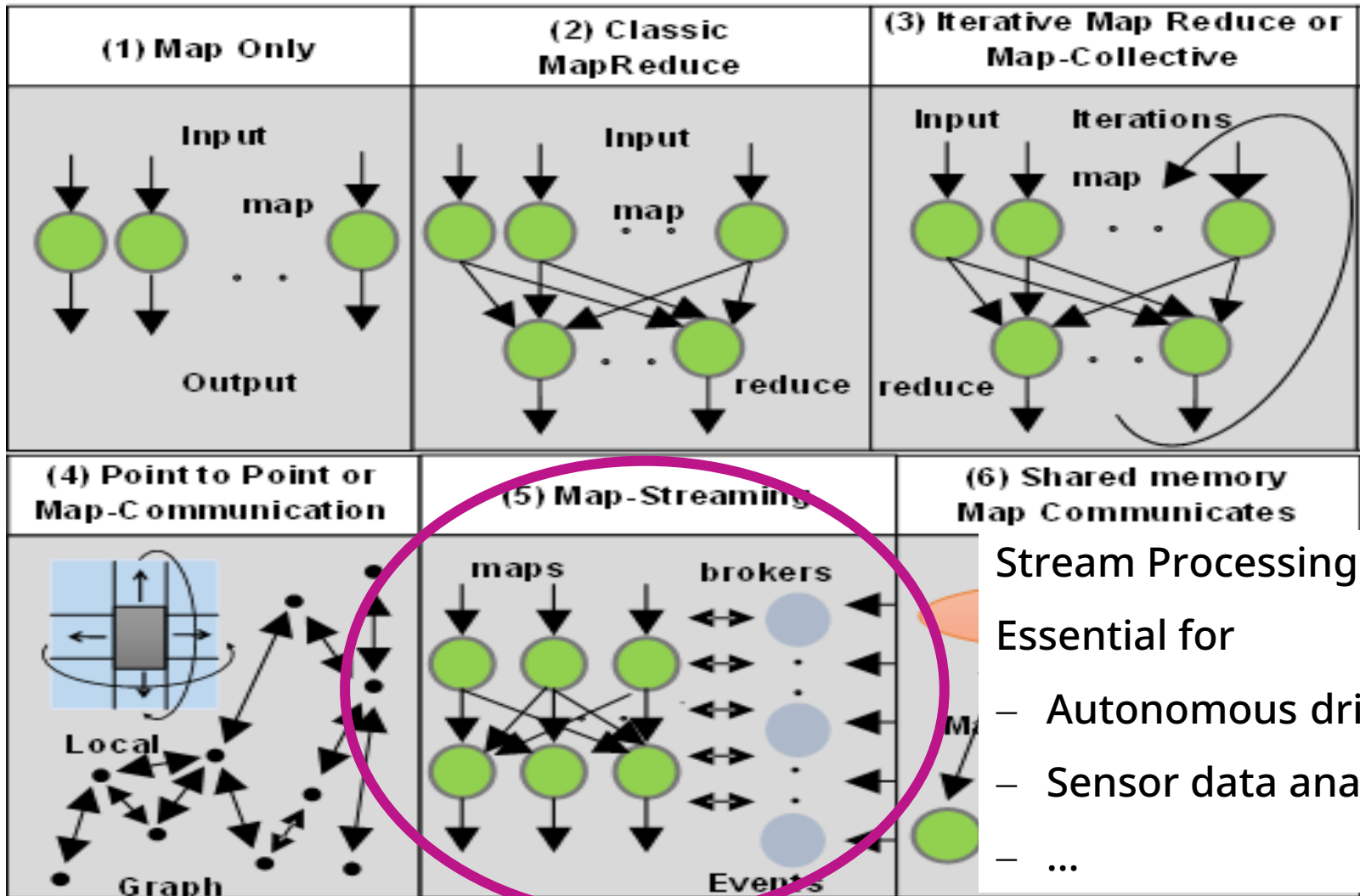
Since Version. 1.4

Memory Management and Binary Processing: leveraging application semantics to manage memory explicitly and eliminate the overhead of JVM object model and garbage collection

Cache-aware computation: algorithms and data structures to exploit memory hierarchy

Code generation: using code generation to exploit modern compilers and CPUs

Limitations and Extensions to MapReduce



Stream Processing:

Essential for

- Autonomous driving
- Sensor data analysis
- ...

Geoffrey Fox: Lecture: Building a Library at the Nexus of High Performance Computing and Big Data, Indiana University

Big data streaming

Availability of sensors & cyber physical Systems



Big data streaming – use case traffic monitoring

Wide range of applications:

- Toll collection, Speed control, traffic-jam detection, route planning

Data collection:

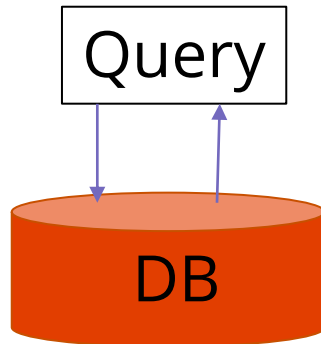
- Toll stations (fixed), smart phones, logistics tracking (GPS)



Stream Processing Model

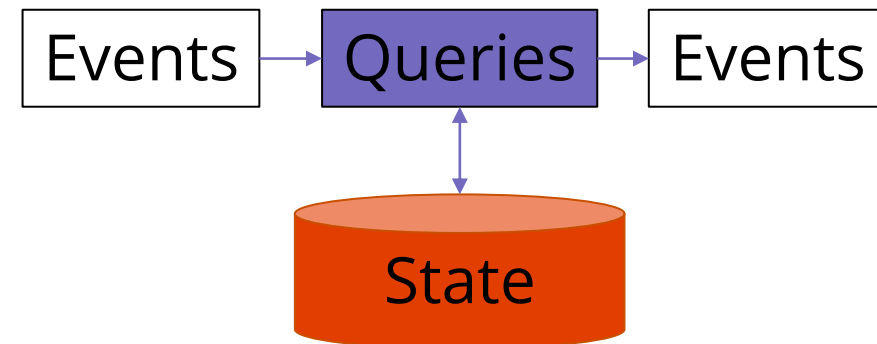
Batch Processing

- Data is „fixed“
- Workload of queries run into the system



Stream Processing

- Continuous (standing) query
- Data is streaming through the system
- Unbounded stream of data
- Limited resources for state



Stream Processing Alternatives



Apache Storm

- True streaming, low latency - lower throughput
- Low level API (Bolts, Spouts) + Trident



Spark Streaming

- Stream processing on top of batch system, high throughput - higher latency
- Functional API (DStreams), restricted by batch runtime



Apache Samza

- True streaming built on top of Apache Kafka, state is first class citizen
- Slightly different stream notion, low level API



Apache Flink

- True streaming with adjustable latency-throughput trade-off
- Rich functional API exploiting streaming runtime; e.g. rich windowing semantics

Apache Flink

Large-scale data processing engine – similar to Spark

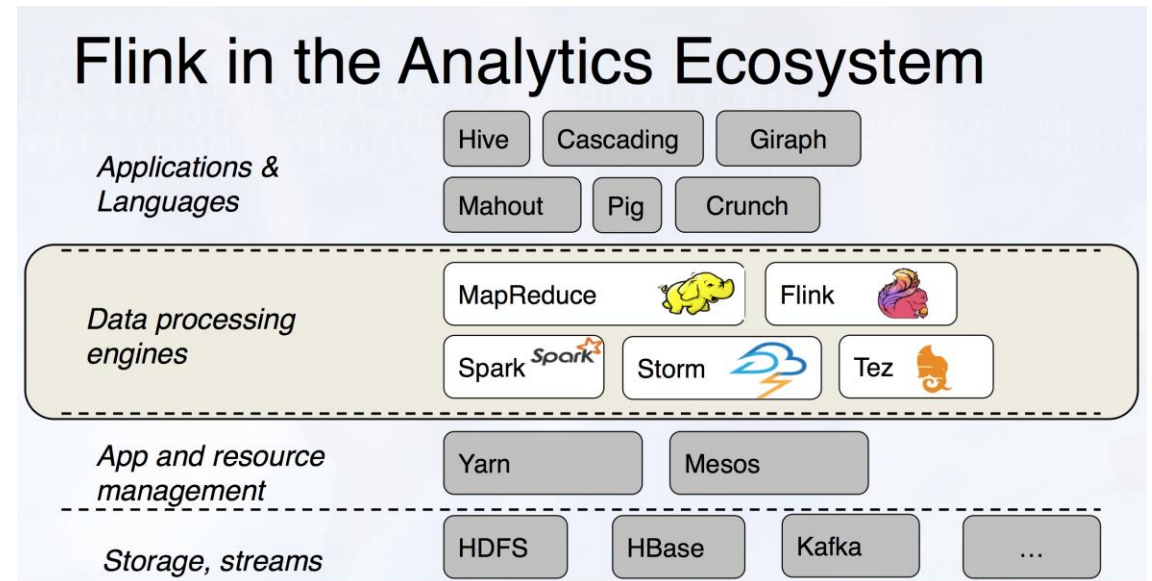
Derived from early developments in Stratosphere project

Many similarities to Apache Spark

Support for iterations (machine learning), graph and batch processing

Good support for streaming

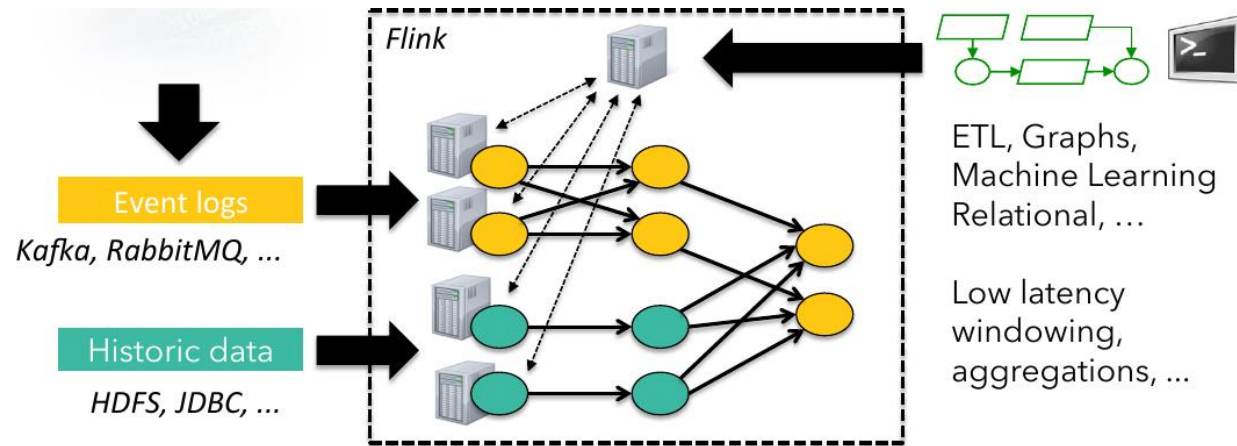
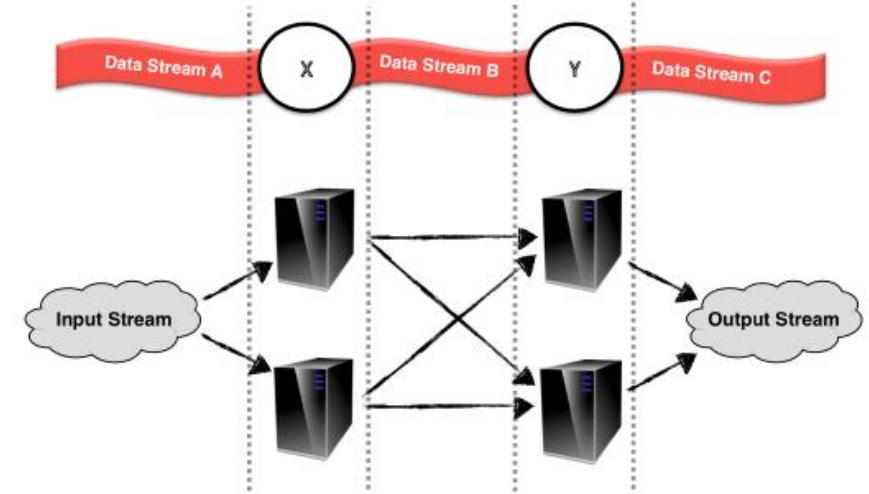
Streaming of results between operators



source: Dr. Tilmann Rabl

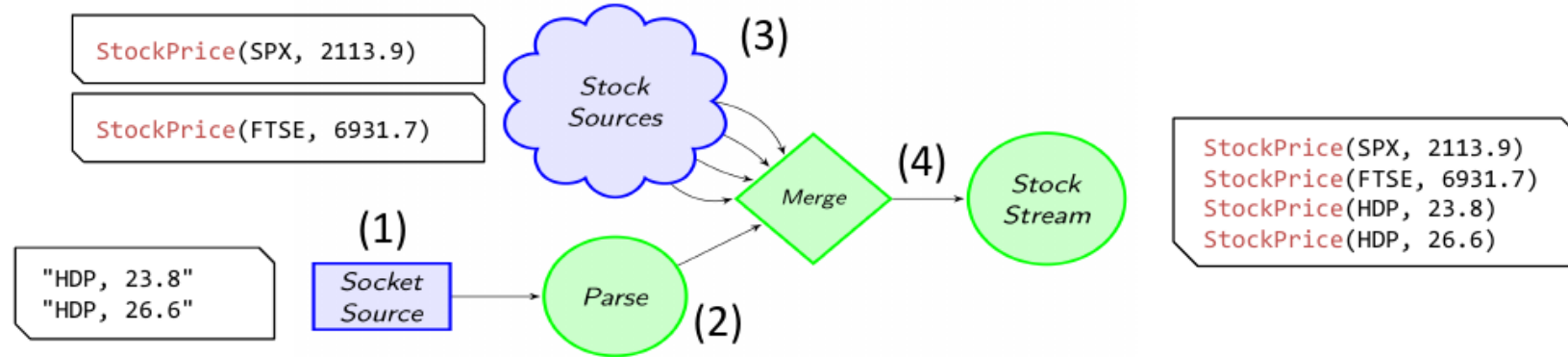
Stream Processing: Apache Flink

- True streaming with adjustable latency and throughput
- Rich functional API exploiting streaming runtime
- Flexible windowing semantics
- Exactly-once processing guarantees with (small) state
- Data stream outputs

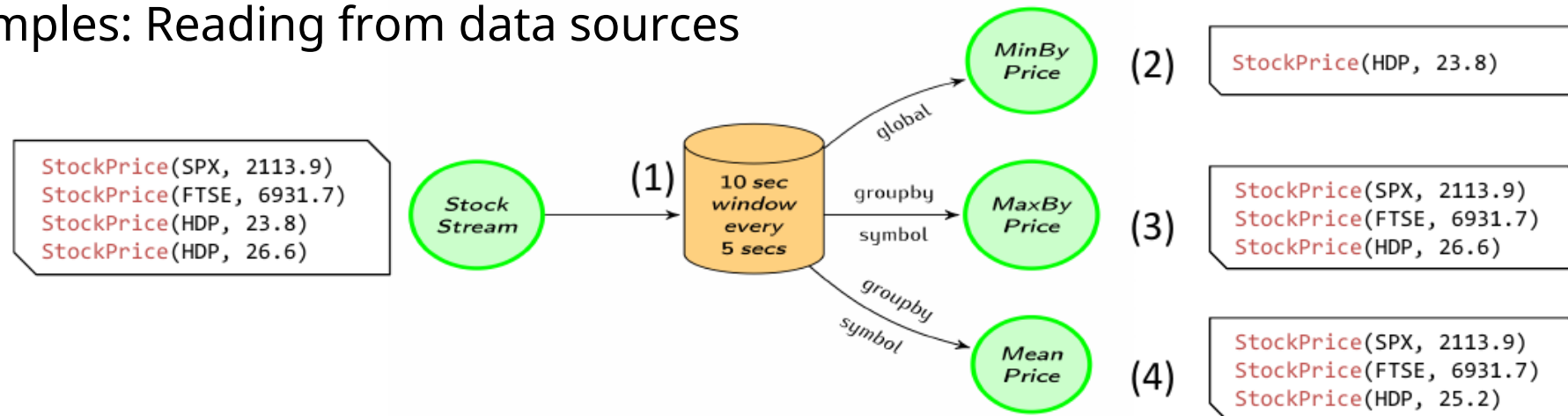


Marton Balassi – data Artisans: «Real-time Stream Processing with Apache Flink»

Stream Processing: Apache Flink - Example



Scala examples: Reading from data sources



Spark Streaming

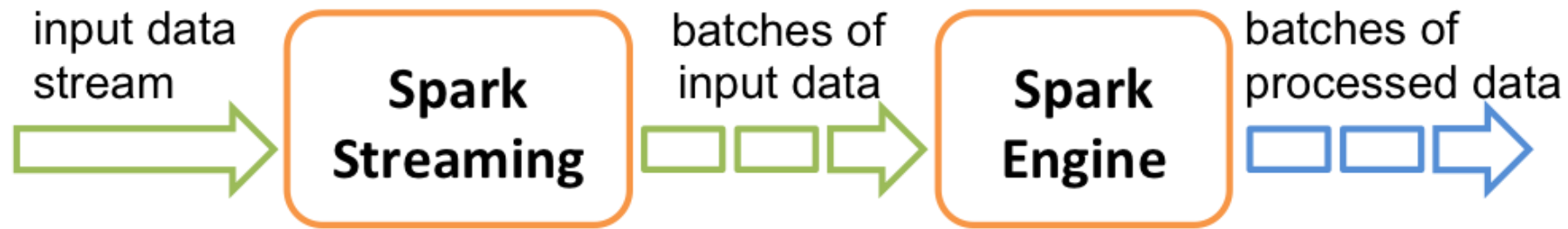
API to extend core towards streaming



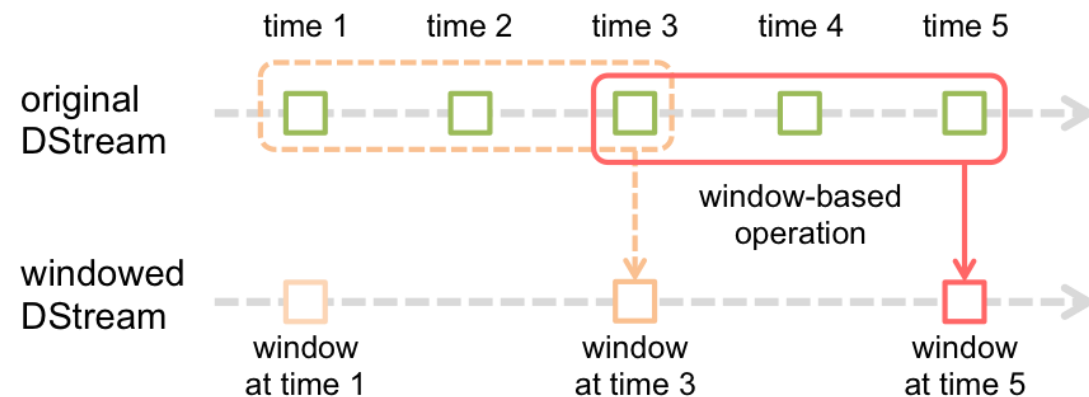
<https://spark.apache.org/docs/latest/streaming-programming-guide.html>

Spark Streaming

API to extend core towards streaming



Input is split in so called mini batches
Also window semantics introduced



<https://spark.apache.org/docs/latest/streaming-programming-guide.html>

Spark vs. Flink Streaming

Flink: direct API to access events in real streaming attempt

Spark: Input is split in so called mini batches but streaming is mapped on processes

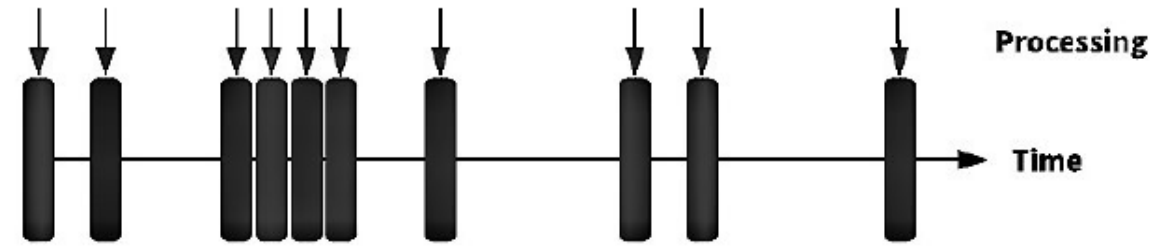


Fig. 1. Data Stream Processing Flow: Native Streaming

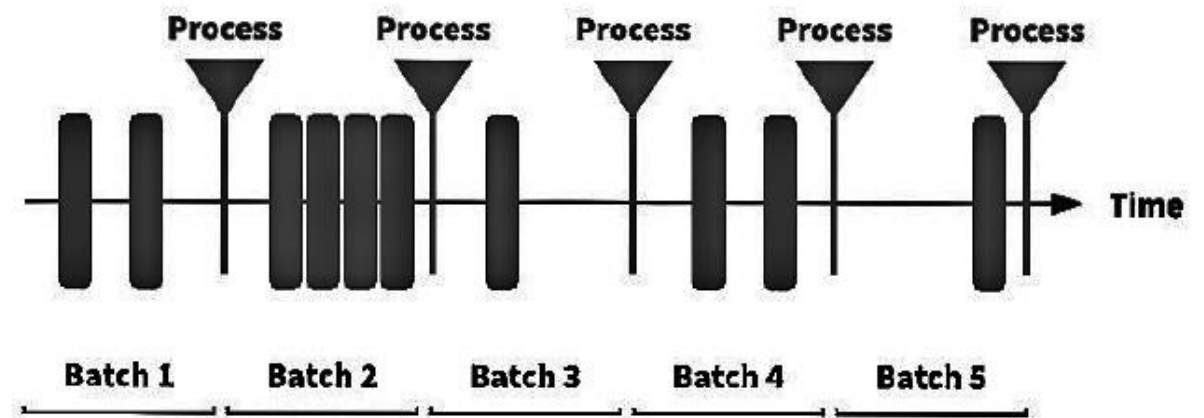


Fig. 2. Data Stream Processing Flow: Micro-batching

Gorasiya, Darshankumar. (2019). Comparison of Open-Source Data Stream Processing Engines: Spark Streaming, Flink and Storm. 10.13140/RG.2.2.16747.49440.

Spark vs. Flink Streaming

Performance Estimate:
windowed aggregation

Comparison of Storm, Spark and
Flink implementations
for varying data stream size
(up to maximum throughput)

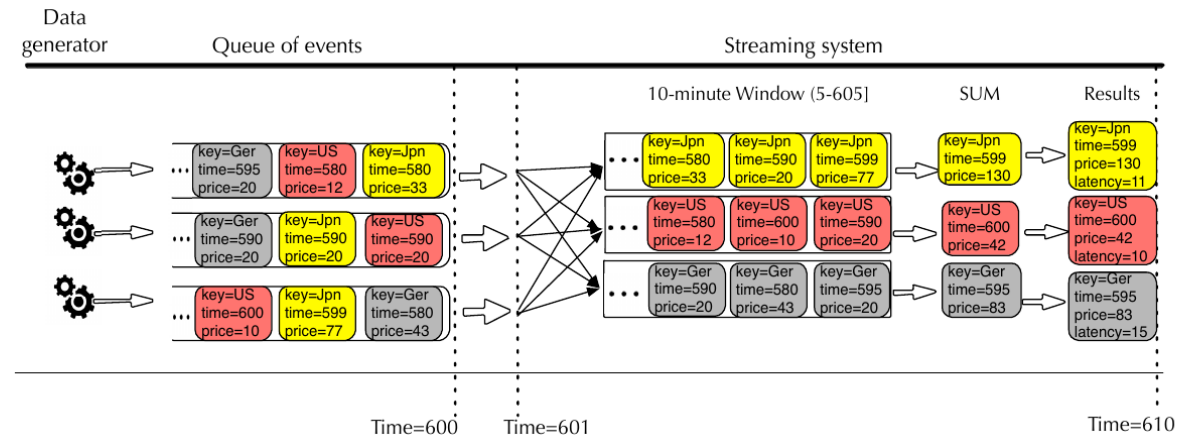


Fig. 1: End-to-end example of an aggregation query. The data generator produces tuples on a given event-time (before time=600), a second later the events enter the streaming system where they are grouped by their key, and finally aggregated (SUM). The event-time latency of the output tuple equals to the maximum event-time latency of elements in each window.

J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen and V. Markl, "Benchmarking Distributed Stream Data Processing Systems," 2018 IEEE 34th International Conference on Data Engineering (ICDE), Paris, 2018, pp. 1507-1518, doi: 10.1109/ICDE.2018.00169.

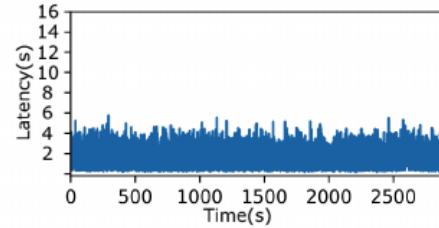
Spark vs. Flink Streaming

Performance Estimate:
windowed aggregation

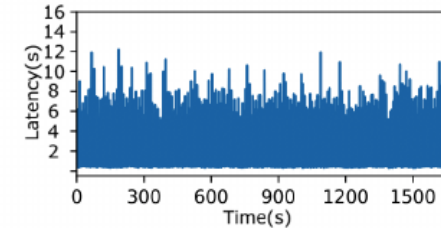
Comparison of Storm, Spark and
Flink implementations for varying
data stream size (up to
maximum throughput)

Results: good scaling with
Spark & Flink

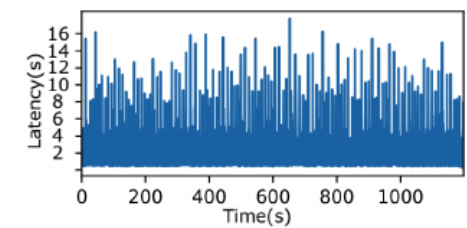
Slight offset for Spark



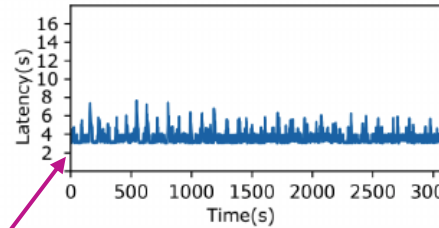
(a) Storm, 2-node, max throughput



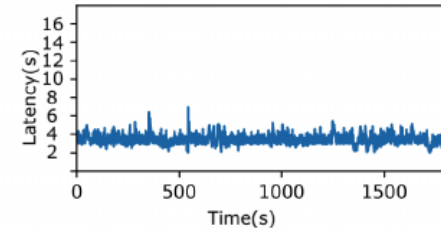
(b) Storm, 4-node, max throughput



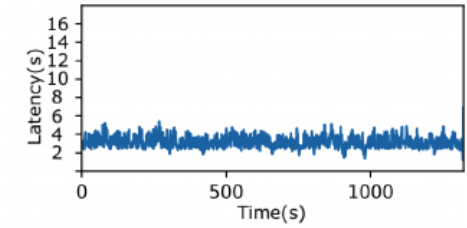
(c) Storm, 8-node, max throughput



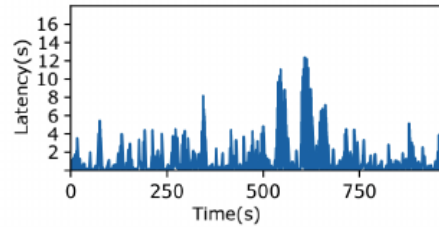
(d) Spark, 2-node, max throughput



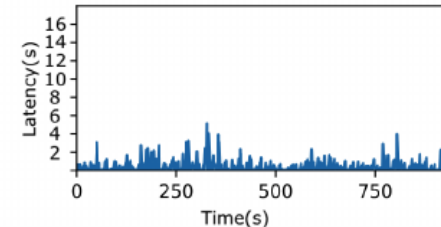
(e) Spark, 4-node, max throughput



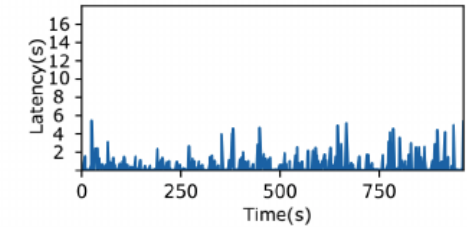
(f) Spark, 8-node, max throughput



(g) Flink, 2-node, max throughput



(h) Flink, 4-node, max throughput



(i) Flink, 8-node, max throughput

J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen and V. Markl, "Benchmarking Distributed Stream Data Processing Systems," 2018 IEEE 34th International Conference on Data Engineering (ICDE), Paris, 2018, pp. 1507-1518, doi: 10.1109/ICDE.2018.00169.

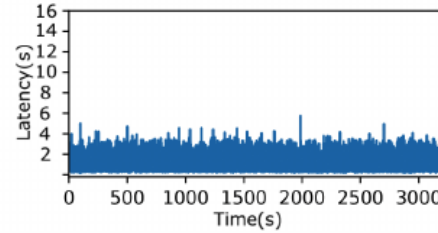
Spark vs. Flink Streaming

Performance Estimate:
windowed aggregation

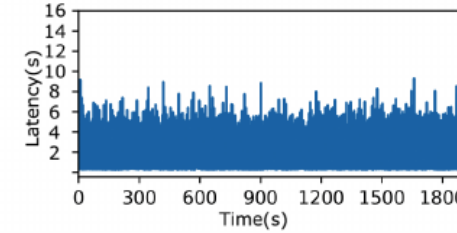
Comparison of Storm, Spark and
Flink implementations for
varying data stream size
(**90% throughput**)

Results: good scaling with
Spark & Flink

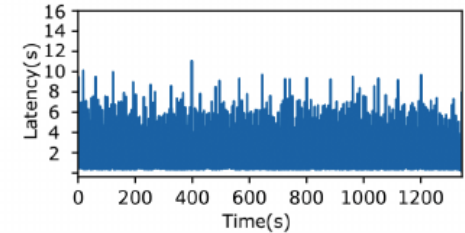
Slight offset for Spark



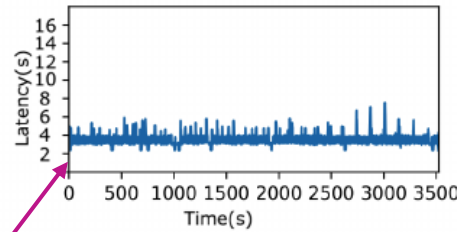
(j) Storm, 2-node, 90%- throughput



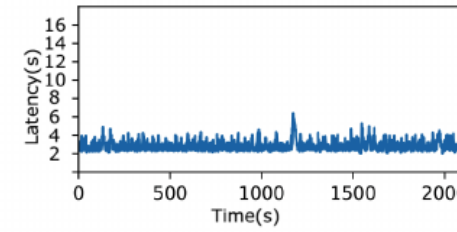
(k) Storm, 4-node, 90%- throughput



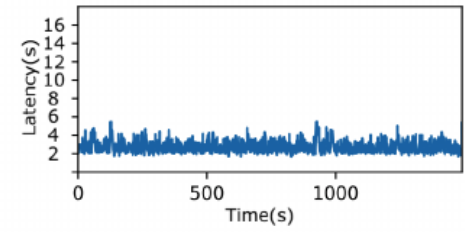
(l) Storm, 8-node, 90%- throughput



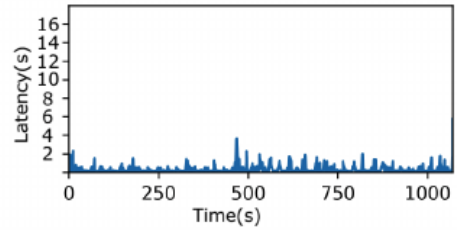
(m) Spark, 2-node, 90%- throughput



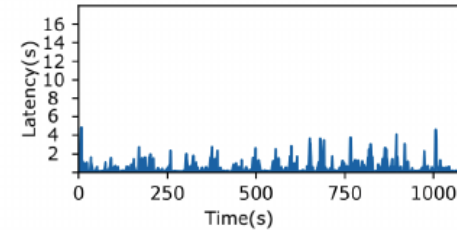
(n) Spark, 4-node, 90%- throughput



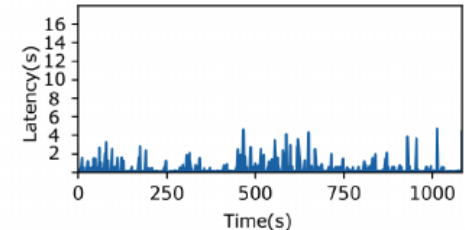
(o) Spark, 8-node, 90%- throughput



(p) Flink, 2-node, 90%- throughput



(q) Flink, 4-node, 90%- throughput



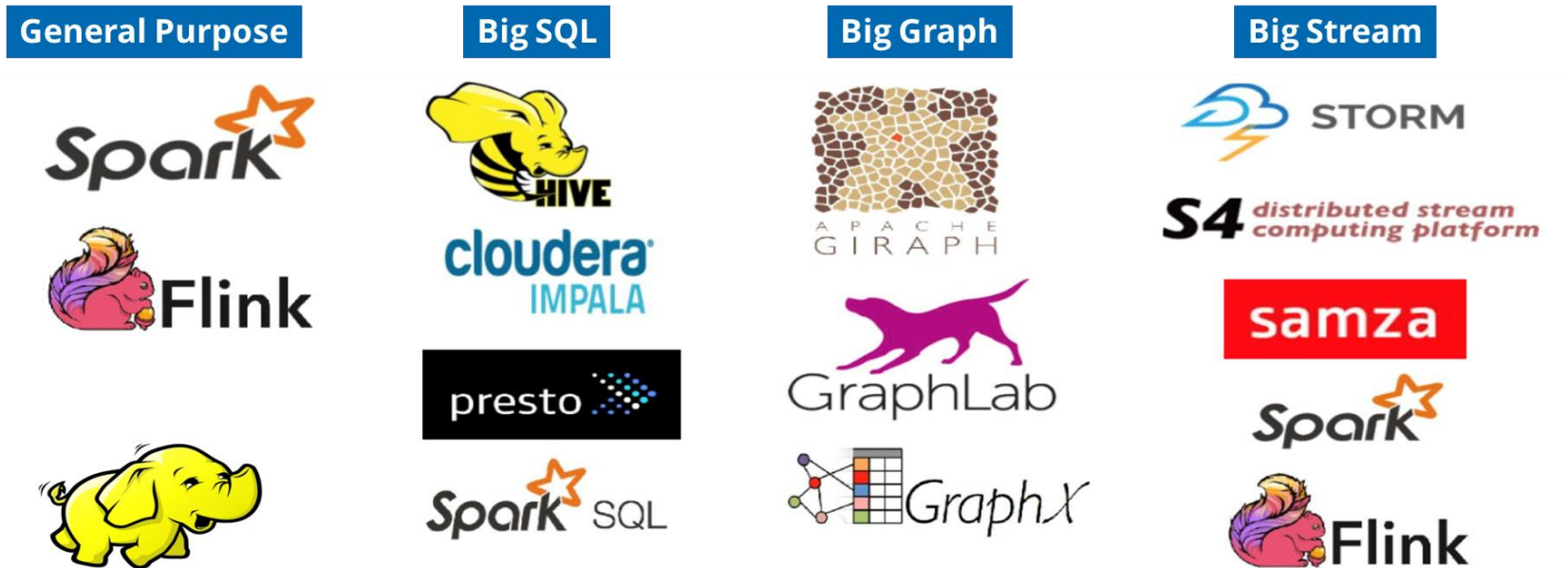
(r) Flink, 8-node, 90% throughput

J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen and V. Markl, "Benchmarking Distributed Stream Data Processing Systems," 2018 IEEE 34th International Conference on Data Engineering (ICDE), Paris, 2018, pp. 1507-1518, doi: 10.1109/ICDE.2018.00169.

Disclaimer

Although we have concentrated on Apache Spark/Flink in the discussions here, that does not mean that this particular framework is best

There are a lot of competing frameworks and specialized approaches out there...



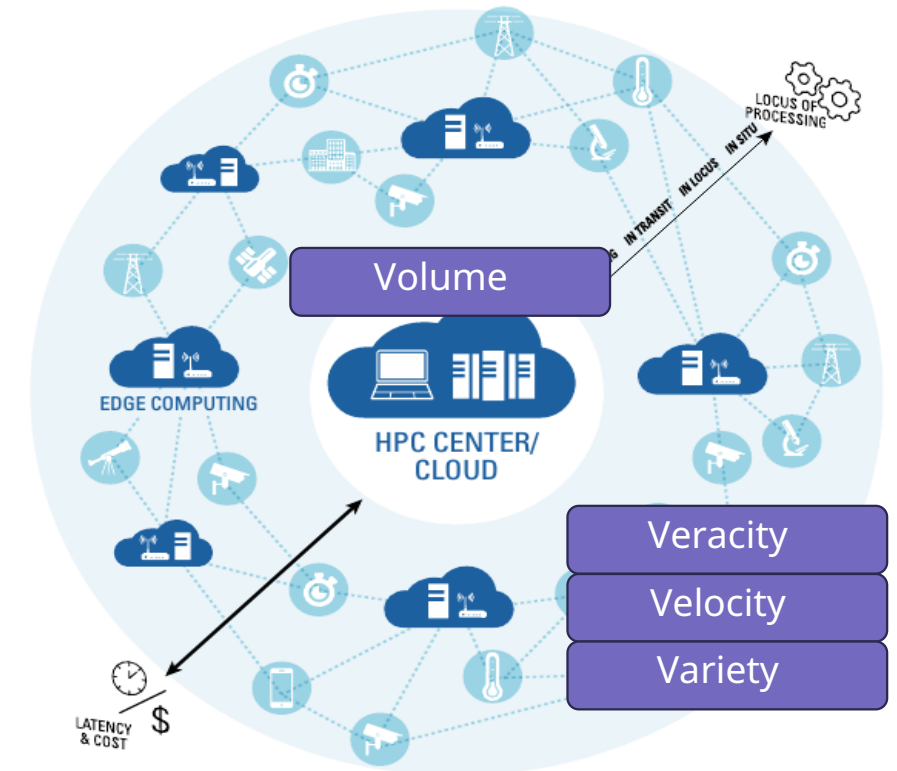
Scheduling

Convergence patterns of HPC and Big Data



From the application/users perspective:

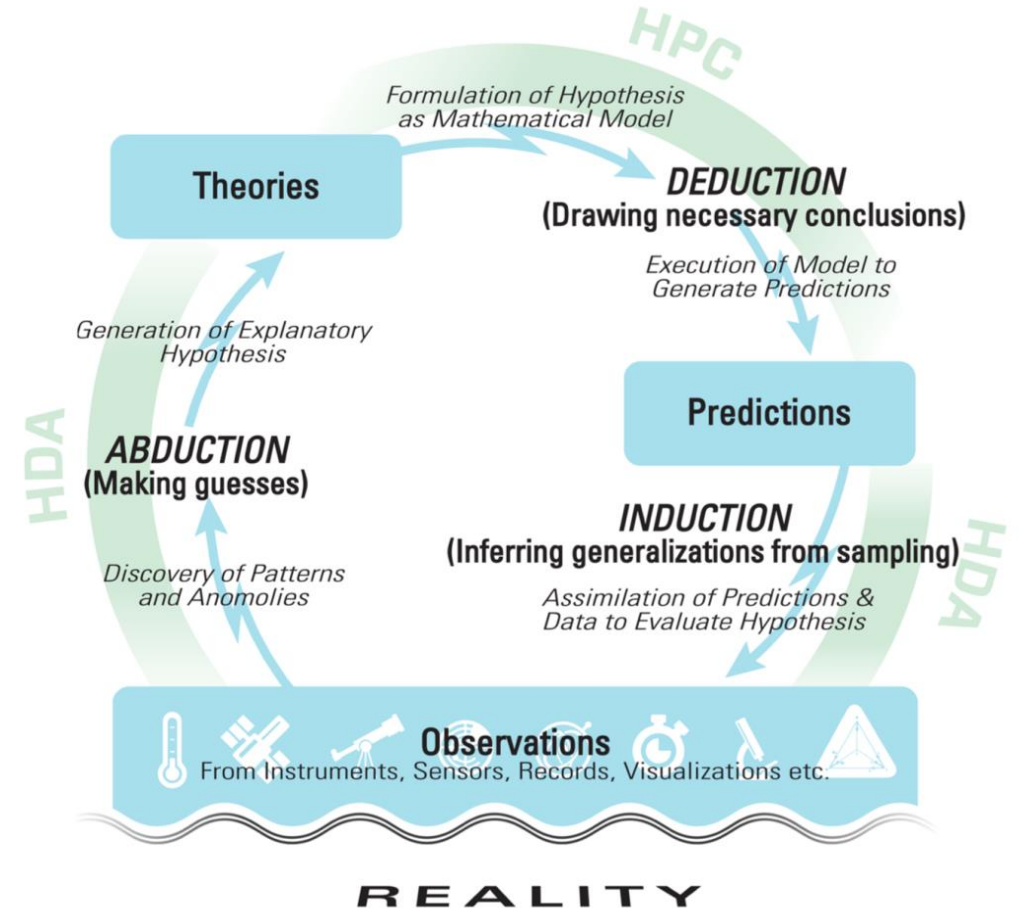
- Traditionally, data and compute is in rather close proximity, e.g. at the data center; not the case in cloud settings
- “Data concentration” gets reduced towards edges
- Developments driven by big data aspects “velocity”, “veracity” and “variety” (heterogeneous data)
- Dealing with the data “deluge” is harder at the edges of the data space
- Edge devices may produce high data rates: e.g. microscopes, gene sequencers, sensor networks, ...



Convergence patterns of HPC and Big Data

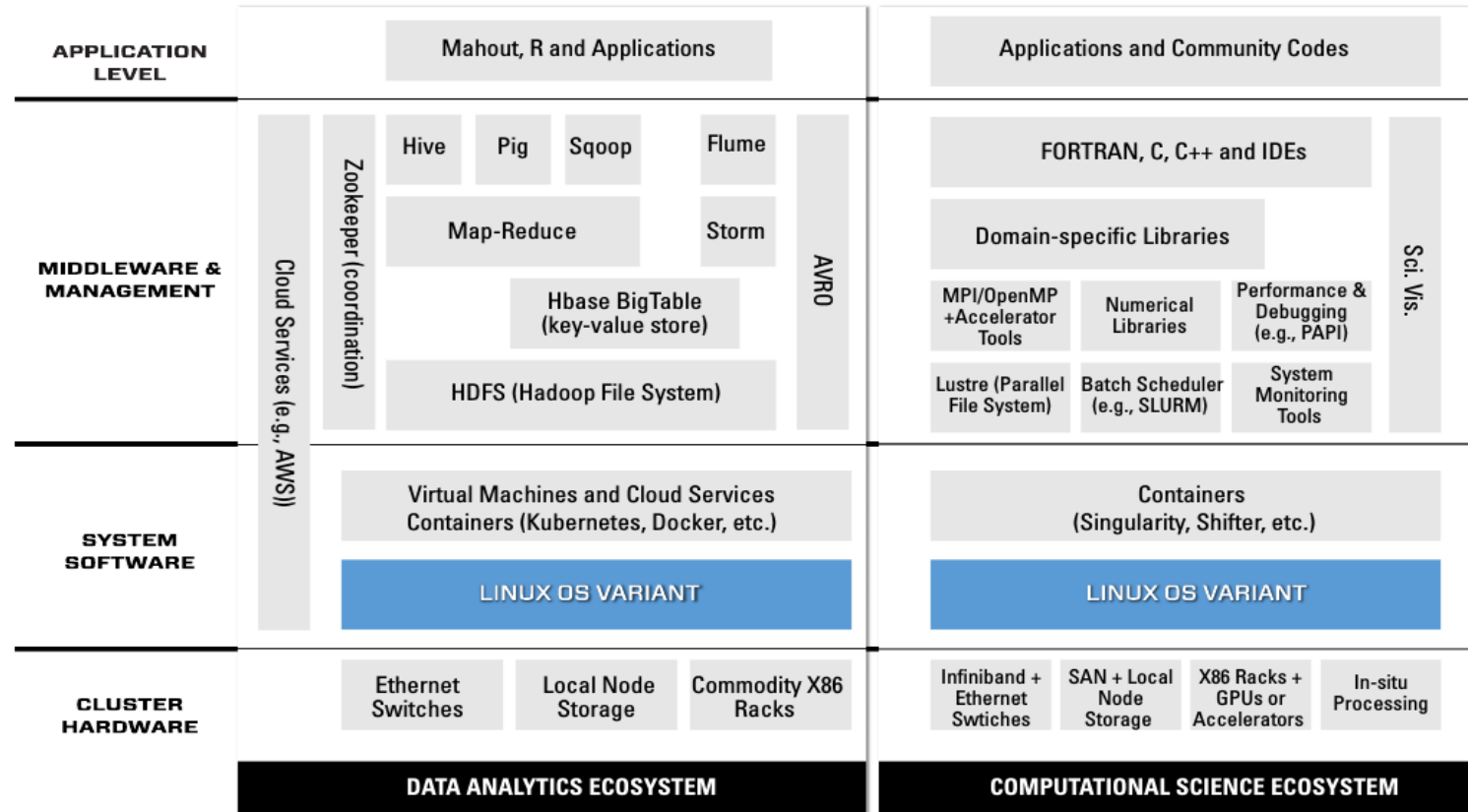
From the application/users perspective:

- HPC + HDA (High-end Data Analytics) workflows
- Analytics scenarios are not singular applications, but rather complex workflows
- Need to cover full data life cycle
- Need „user in the loop“: confirm model iterations or outcome of predictive analytics
- Potentially highly iterative
- Flexible enough to react on changes in software stack
 - Interoperability of tools
 - Reproducibility of results
 - Adaptability to systems



Software stacks of Big Data and HPC

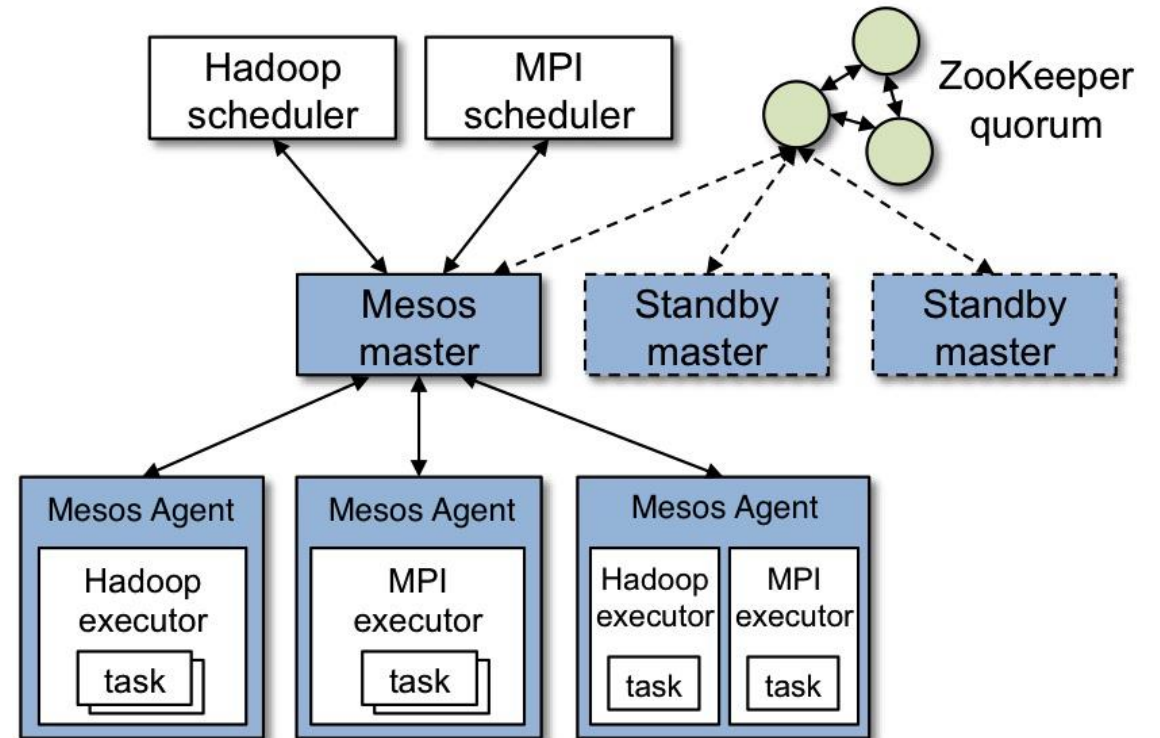
From the software ecosystems perspective: The past and current status



Big Data and HPC – convergence patterns: scheduler evolution

Support Big Data workloads on HPC

- Big data framework such as Hadoop are not just data intensive applications, but can be seen as more service oriented frameworks
- Direct comparison to classic MPI jobs misleading
- Abstract scheduling out of big data framework in general way to allow more flexible hardware provisioning and usage, E.g. Apache Mesos:
 - Further developments of schedulers required, active research field, e.g. Apache Mesos as abstract scheduler



Big Data and HPC – convergence patterns: scheduler evolution

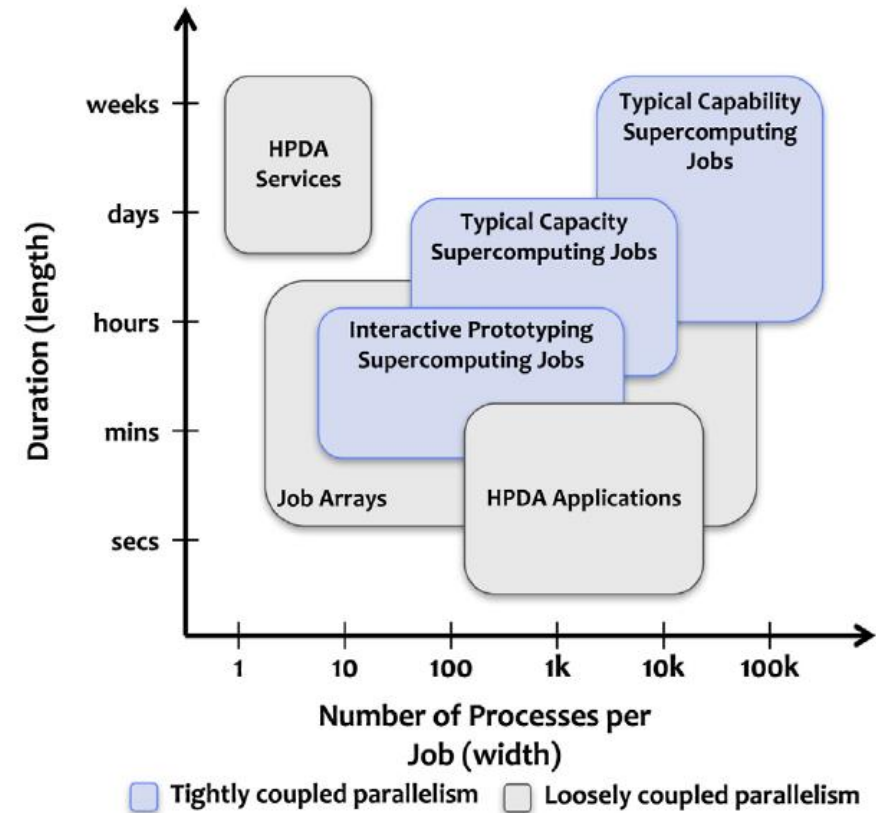
Today's HPC systems are faced with diverse workloads

HPDA (High Performance Data Analytics) services and applications

Capacity supercomputing jobs (simulations)

Interactive Prototyping

Job chains: mapping dependencies between workflow steps



<https://www.sciencedirect.com/science/article/pii/S0743731517301983?via%3Dihub>

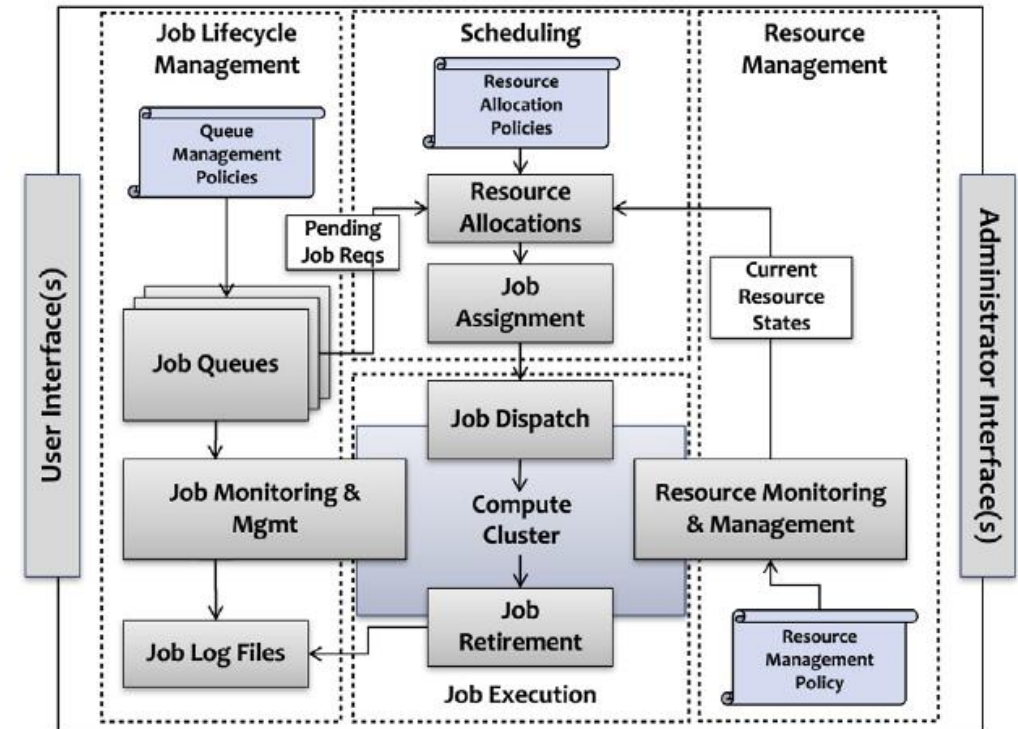
Big Data and HPC – convergence patterns: scheduler evolution

Today's HPC systems are faced with diverse management requirements

Scheduling layer: manages resources from user requirements

Submission queues influenced by resource allocation policies, but also by management and job lifecycle aspects

Tension between usability ('easy to use') and management perspective

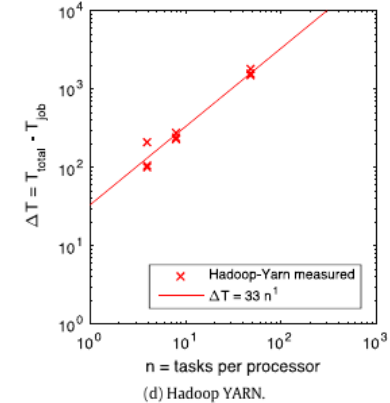
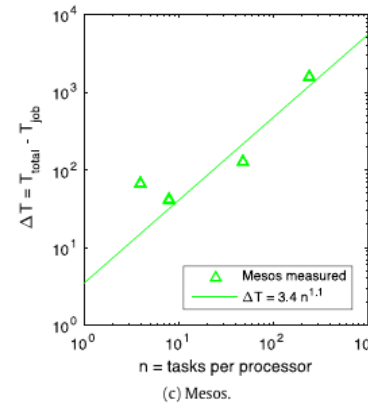
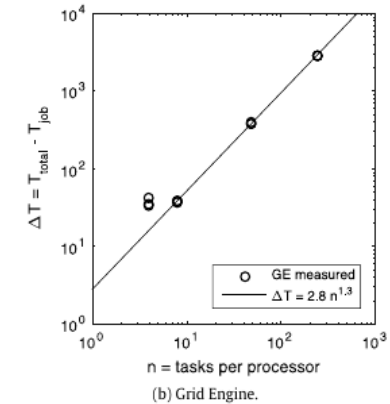
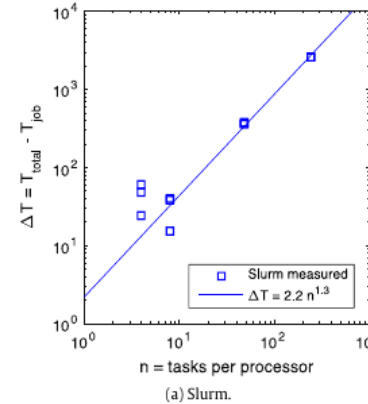


<https://www.sciencedirect.com/science/article/pii/S0743731517301983?via%3Dihub>

Big Data and HPC – convergence patterns: scheduler evolution

Comparison of major state-of-the-art schedules

- Slurm vs. Grid Engine vs. Mesos vs. Yarn
- Modern schedules up to factor 10 faster in task scheduling, concerning overhead ($T_{\text{total}} - T_{\text{job}}$)
- Important in case of many small jobs, or long process chains in iterative analysis



<https://www.sciencedirect.com/science/article/pii/S0743731517301983?via%3Dihub>

Big Data Analytics @HPC – HPDA (high performance data analytics)

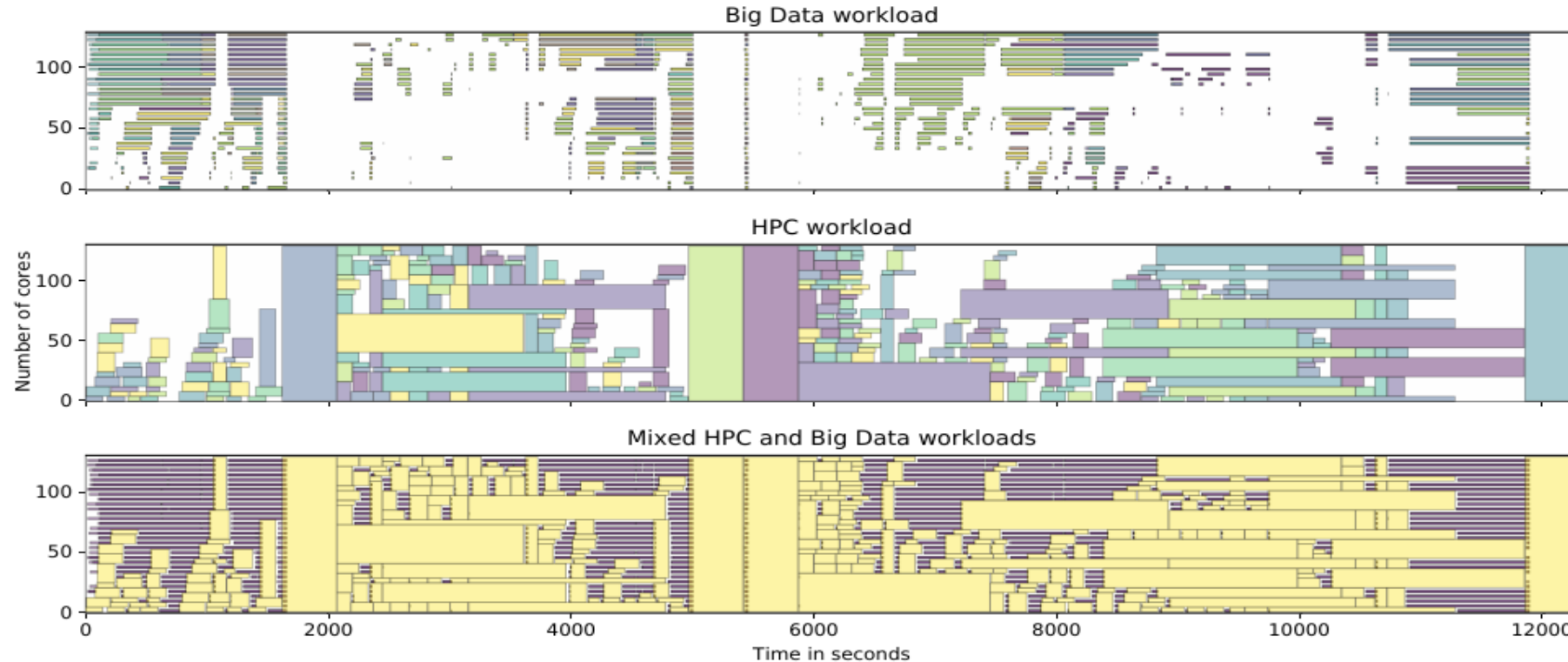
Ongoing research to support different jobs and system usage on HPC architectures

Why using one scheduler?

Highly parallel HPC workloads require large parts of the system to be freed prior to scheduling

In principle theses 'holes' can be filled with small and not time depended analytics jobs

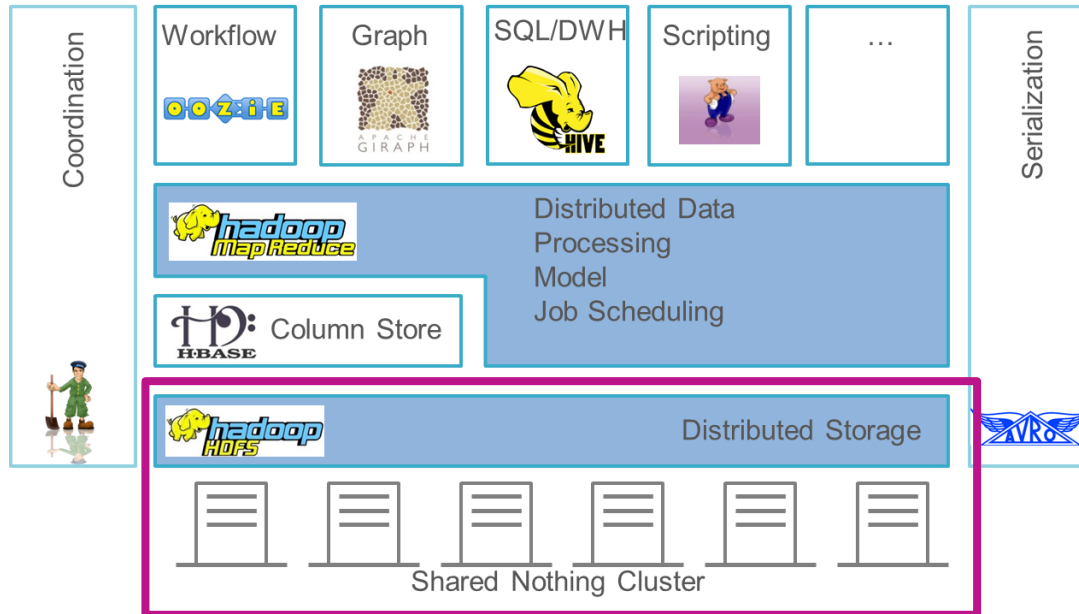
Big Data Analytics @HPC



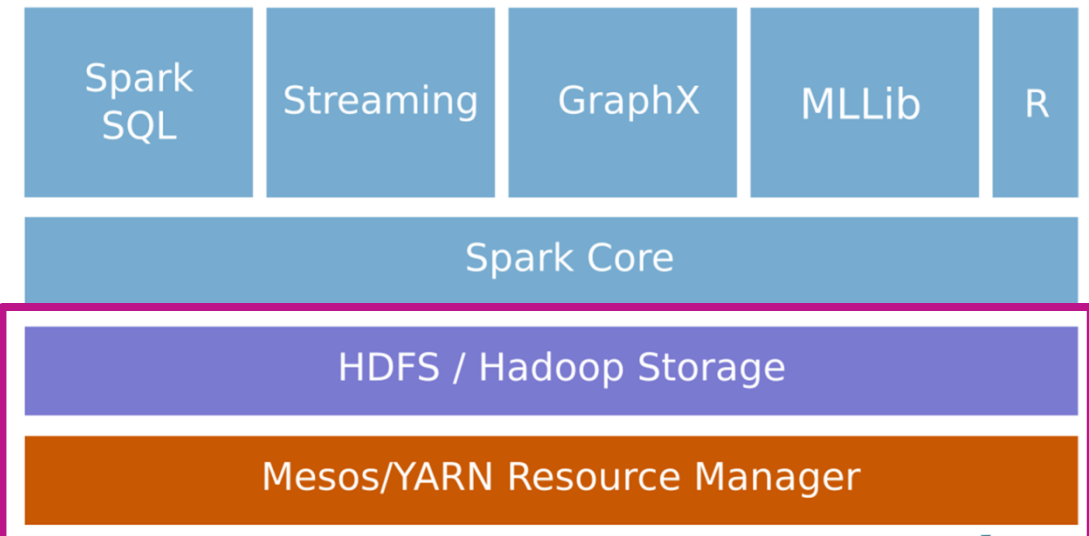
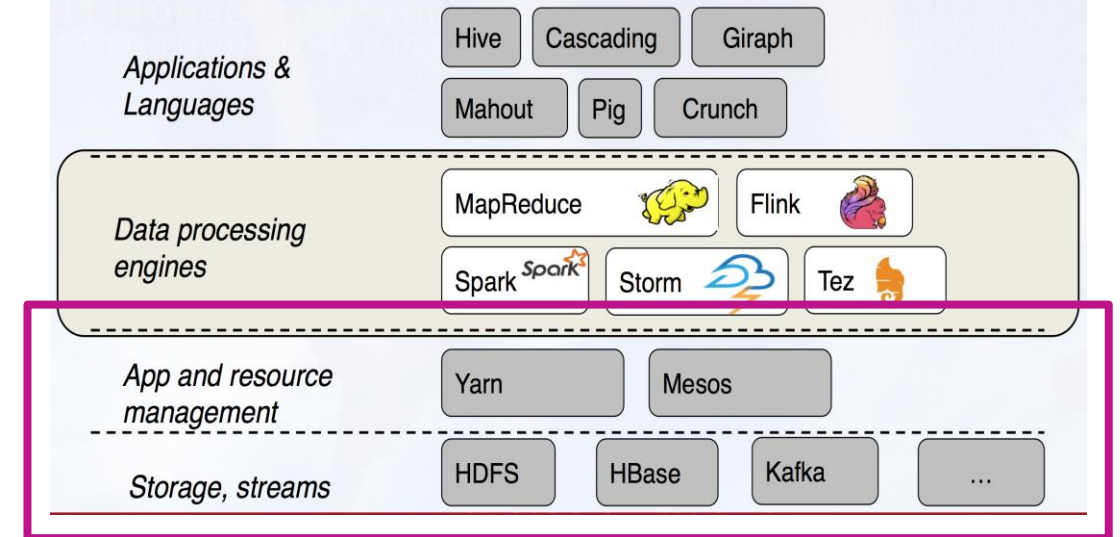
Ongoing research to support different jobs and system usage on HPC architectures

...

M. Mercier, D. Glessner, Y. Georgiou and O. Richard, "Big data and HPC collocation: Using HPC idle resources for Big Data analytics," 2017 IEEE International Conference on Big Data (Big Data), Boston, MA, 2017, pp. 347-352.
doi: 10.1109/BigData.2017.8257944



Flink in the Analytics Ecosystem



Summary

Constantly evolving software frameworks to support analytics scenarios

Incorporating more hardware capabilities: in-memory processing

Reflection on more demand for stream processing

Development of extended software ecosystem for data analytics, e.g. Spark, Flink as successor project to Hadoop/MapReduce

Nowadays more convergence of Big Data and HPC ecosystem